

# Reliable extrapolation of deep neural operators informed by physics or sparse observations

Min Zhu<sup>a</sup>, Handi Zhang<sup>b</sup>, Anran Jiao<sup>c</sup>, George Em Karniadakis<sup>d,e</sup>, Lu Lu<sup>a,\*</sup>

<sup>a</sup> Department of Chemical and Biomolecular Engineering, University of Pennsylvania, Philadelphia, PA 19104, USA

<sup>b</sup> Graduate Group in Applied Mathematics and Computational Science, University of Pennsylvania, Philadelphia, PA 19104, USA

<sup>c</sup> Department of Computer and Information Science, University of Pennsylvania, Philadelphia, PA 19104, USA

<sup>d</sup> Division of Applied Mathematics, Brown University, Providence, RI 02912, USA

<sup>e</sup> School of Engineering, Brown University, Providence, RI 02912, USA

Received 9 February 2023; received in revised form 11 April 2023; accepted 12 April 2023

Available online 2 May 2023

## Abstract

Deep neural operators can learn nonlinear mappings between infinite-dimensional function spaces via deep neural networks. As promising surrogate solvers of partial differential equations (PDEs) for real-time prediction, deep neural operators such as deep operator networks (DeepONets) provide a new simulation paradigm in science and engineering. Pure data-driven neural operators and deep learning models, in general, are usually limited to interpolation scenarios, where new predictions utilize inputs within the support of the training set. However, in the inference stage of real-world applications, the input may lie outside the support, i.e., extrapolation is required, which may result to large errors and unavoidable failure of deep learning models. Here, we address this challenge of extrapolation for deep neural operators. First, we systematically investigate the extrapolation behavior of DeepONets by quantifying the extrapolation complexity, via the 2-Wasserstein distance between two function spaces and propose a new strategy of bias–variance trade-off for extrapolation with respect to model capacity. Subsequently, we develop a complete workflow, including extrapolation determination, and we propose five reliable learning methods that guarantee a safe prediction under extrapolation by requiring additional information—the governing PDEs of the system or sparse new observations. The proposed methods are based on either fine-tuning a pre-trained DeepONet or multifidelity learning. We demonstrate the effectiveness of the proposed framework for various types of parametric PDEs. Our systematic comparisons provide practical guidelines for selecting a proper extrapolation method depending on the available information, desired accuracy, and required inference speed.

© 2023 Elsevier B.V. All rights reserved.

**Keywords:** Neural operators; DeepONet; Extrapolation complexity; Fine-tuning; Multifidelity learning; Out-of-distribution inference

## 1. Introduction

The universal approximation theorem of neural networks (NNs) for functions [1] has provided a rigorous foundation of deep learning. As an increasingly popular alternative to traditional numerical methods such as finite difference and finite element methods, neural networks have been applied in solving partial differential equations

\* Corresponding author.

E-mail address: [lulu1@seas.upenn.edu](mailto:lulu1@seas.upenn.edu) (L. Lu).

(PDEs) in the field of scientific machine learning (SciML) [2,3]. Physics-informed neural networks (PINNs) [4,5] have provided a new paradigm for solving forward as well as inverse problems [6] governed by PDEs by embedding the PDE loss into the loss function of neural networks.

Neural networks are universal approximators of not only functions but also nonlinear operators, i.e., mappings between infinite-dimensional function spaces [7–9]. Hence, NNs can approximate the operators of PDEs, and once the network is trained, it only requires a forward pass to obtain the PDE solution for a new condition. The deep operator network (DeepONet) [8], the first neural operator, has demonstrated good performance for building surrogate models for many types of PDEs. For example, DeepONet has been applied in multiscale bubble dynamics [10,11], brittle fracture analysis [12], instabilities in boundary layers [13], solar–thermal systems forecasting [14], electroconvection [15], hypersonics with chemical reactions [16], and fast multiscale modeling [17]. In addition, several extensions of DeepONet have been proposed in recent studies, including DeepONet for multiple-input operators (MIONet) [18], DeepONet with proper orthogonal decomposition (POD-DeepONet) [19], physics-informed DeepONet [12,20], multifidelity DeepONet [21–23], DeepM&Mnet for multiphysics problems [15,16], multiscale DeepONet [24], and DeepONet with uncertainty quantification [25–28].

Despite the aforementioned success, neural networks are usually limited to solving interpolation problems, i.e., inference is accurate only for inputs within the support of the training set, while NNs would fail if the new input is outside the support of the training set (i.e., extrapolation) [29,30]. This difficulty of extrapolation has also been observed in DeepONets [8,10,16,17,31,32]. Specifically, we consider to use DeepONet for learning an operator  $\mathcal{G} : v(x) \in \mathcal{V} \mapsto u(\xi) \in \mathcal{U}$ , a map between two function spaces (an input function space  $\mathcal{V}$  and an output function space  $\mathcal{U}$ ). The input function space in general is problem dependent, and the most commonly used space is Gaussian random fields (GRFs; or Gaussian process (GP)). In this study, we limit ourselves to GRFs so that we can define a metric (i.e., Wasserstein distance) in our analysis.

We usually generate a training dataset by randomly sampling input functions  $v$  from a space of mean-zero GRF with a predefined covariance kernel  $k_l(x_1, x_2)$ :

$$v(x) \sim \mathcal{GP}(0, k_l(x_1, x_2)).$$

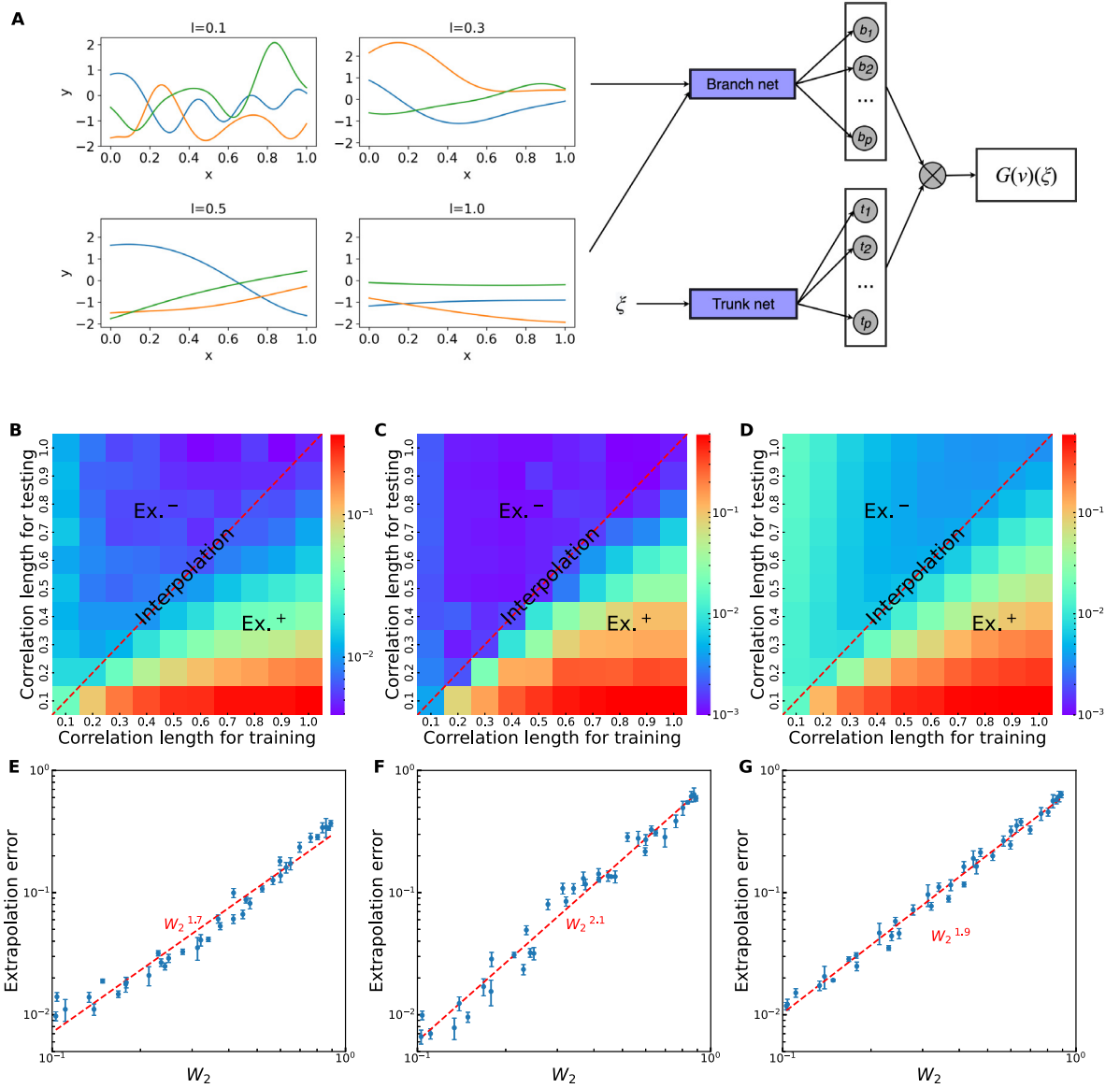
A typical choice of the kernel is the radial-basis function kernel (or squared-exponential kernel, Gaussian kernel)

$$k_l(x_1, x_2) = \exp\left(-\frac{\|x_1 - x_2\|^2}{2l^2}\right), \quad (1)$$

where  $l > 0$  is the correlation length. The value of  $l$  determines the smoothness of sampled functions, with a larger  $l$  leading to smoother functions. Several functions randomly sampled from the spaces of GRFs with different  $l$  are shown in Fig. 1A. If the functions used for training and testing are sampled from the same GRF, then it is interpolation. If we train the DeepONet with smooth functions (e.g., GRF with  $l = 1.0$  in Fig. 1A) and test it with less smooth functions (e.g., GRF with  $l = 0.1$  in Fig. 1A), this is one form of extrapolation, and intuitively this would lead to a large testing error. If the training functions are less smooth than testing functions, this is another form of extrapolation.

In real applications, there is no guarantee that in the inference stage a new input would always be in the interpolation regime, and extrapolation could lead to large errors and possible failure of NNs. In this study, our first goal is to understand the extrapolation of DeepONets. We quantitatively measure the extrapolation complexity via the 2-Wasserstein distance between two function spaces when the correlation length of the function space for training is larger than that of testing. Subsequently, we systematically study how the extrapolation error of DeepONet changes with respect to multiple factors, including model capacity (e.g., network size and training iterations), training dataset size, and activation functions.

In the second part of this work, we develop extrapolation strategies for DeepONets. Given a training dataset, in general, it is almost impossible to have an accurate prediction for a new input outside the support of training dataset with spatio-temporal scales smaller than the training set scales (see the examples of sampled functions in Fig. 1A). In order to reduce the extrapolation error, certain additional information must be added. In this study, we assume that we have one of the following extra information: (1) we know the governing PDE of the system, or (2) we have extra sparse observations at some locations  $\xi$  for the output function  $u$ . An intermediate case is to have available some partial physics with very few observations. Based on the type of information, we propose several methods to solve the extrapolation problem. Specifically, we first train a DeepONet by using the training dataset.



**Fig. 1.** Examples of DeepONets for interpolation and extrapolation. (A) Functions randomly sampled from GRF spaces of different correlation length  $l$  are taken as the input functions for the branch net of DeepONet. (B to G)  $L^2$  relative errors of DeepONet trained and tested on datasets with different correlation lengths. (B and E) ODE problem. (C and F) Diffusion-reaction equation with  $k = 0.01$ . (D and G) Diffusion-reaction equation with  $k = 0.5$ . (B, C and D) Testing error for different pairs of training and testing functions. (E, F and G) The  $\text{Ex.}^+$  error grows with a polynomial rate with respect to the  $W_2$  distance between the training and test spaces. The error is the mean of 10 runs, and the error bars represent one standard deviation.

Next, for the first case with physics, we fine-tune the pre-trained DeepONet with the PDE loss as done in PINNs. For the second case with new observations, we propose to either fine-tune the pre-trained DeepONet with the new data, or train another machine learning model (neural networks or Gaussian process regression) using multifidelity learning [33–37], where the prediction from the pre-trained DeepONet is of low-fidelity while the new data is of high-fidelity.

The idea of using new observations to fine-tune a pre-trained DeepONet for extrapolation has been used in Refs. [10,16]. However, as we show in our numerical experiments, their fine-tuning approach is not always stable and accurate, so here we propose a better approach for fine-tuning. Moreover, fine-tuning a pre-trained network with new data is conceptually related to transfer learning (TL) [38–40] and few-shot learning (FSL) [41,42]. In Ref. [43], a transfer learning technique was developed for DeepONets to solve the same PDEs but on different domains. Although we only consider DeepONets in this study, the proposed methods can directly be applied to other deep neural operators, such as Fourier neural operators [19,44], graph kernel networks [45], nonlocal kernel networks [46], and others [31,47,48].

The paper is organized as follows. In Section 2, after briefly introducing the architecture of DeepONet, we first introduce a definition of the extrapolation complexity and subsequently empirically investigate the extrapolation error with respect to various factors. In Section 3, we provide a general workflow for extrapolation and propose several methods to improve the DeepONet performance for extrapolation, including fine-tuning with physics, fine-tuning with sparse observations, and multifidelity learning. Then in Section 4, we compare the performance of different methods for six benchmarks. Finally, we conclude the paper and discuss some future directions in Section 5.

## 2. Extrapolation of deep neural operators

We briefly explain the architecture of the deep operator network (DeepONet) and then focus on how extrapolation errors vary with respect to different factors, including extrapolation complexity, training phase, training dataset size, and network architecture (e.g., network size and activation function).

### 2.1. Operator learning via DeepONet

To define the setup of operator learning, we consider a function space  $\mathcal{V}$  of function  $v$  defined on the domain  $D \subset \mathbb{R}^d$ :

$$v : D \ni x \mapsto v(x) \in \mathbb{R},$$

and another function space  $\mathcal{U}$  of function  $u$  defined on the domain  $D' \subset \mathbb{R}^{d'}$ .

$$u : D' \ni \xi \mapsto u(\xi) \in \mathbb{R}.$$

Let  $\mathcal{G}$  be an operator that maps  $\mathcal{V}$  to  $\mathcal{U}$ :

$$\mathcal{G} : \mathcal{V} \rightarrow \mathcal{U}, \quad v \mapsto u.$$

DeepONet [8] was developed to learn the operator  $\mathcal{G}$  based on the universal approximation theorem of neural networks for operators [7]. A DeepONet consists of two sub-networks: a trunk network and a branch network (Fig. 1A). For  $m$  scattered locations  $\{x_1, x_2, \dots, x_m\}$  in  $D$ , the branch network takes the function evaluations  $[v(x_1), v(x_2), \dots, v(x_m)]$  as the input, and the output of the branch network is  $[b_1(v), b_2(v), \dots, b_p(v)]$ , where  $p$  is the number of neurons. The trunk network takes  $\xi$  as the input and the outputs  $[t_1(\xi), t_2(\xi), \dots, t_p(\xi)]$ . Then, by taking the inner product of trunk and branch outputs, the output of DeepONet is:

$$\mathcal{G}(v)(\xi) = \sum_{k=1}^p b_k(v) t_k(\xi) + b_0,$$

where  $b_0 \in \mathbb{R}$  is a bias.

### 2.2. Extrapolation testing setup

We consider two examples to demonstrate the extrapolation of DeepONets.

*Antiderivative operator.* The first one is an ordinary differential equation (ODE) defined by

$$\frac{du(x)}{dx} = v(x), \quad x \in [0, 1],$$

with an initial condition  $u(0) = 0$ . We use DeepONet to learn the solution operator, i.e., an antiderivative operator

$$\mathcal{G} : v(x) \mapsto u(x) = \int_0^x v(\tau) d\tau.$$

*Diffusion-reaction equation.* The second example is a diffusion-reaction equation defined by

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + ku^2 + v(x), \quad x \in [0, 1], t \in [0, 1],$$

with zero initial and boundary conditions. In this example,  $k$  and  $D$  are set at 0.01. DeepONet is trained to learn the mapping from the source term  $v(x)$  to the solution  $u(x, t)$ :

$$\mathcal{G} : v(x) \mapsto u(x, t).$$

*Training and test datasets.* The input functions  $v$  are sampled from a mean-zero Gaussian random field (GRF)  $v \sim \mathcal{GP}(0, k_l(x_1, x_2))$  with the radial-basis function (RBF) kernel of Eq. (1). However, the training and test datasets use different values of  $l$ . The reference solution  $u(x)$  of the ODE is obtained by Runge–Kutta(4, 5), and the reference solution  $u(x, t)$  of the PDE is obtained by a second-order finite difference method with a mesh size of  $101 \times 101$ .

### 2.3. Interpolation and extrapolation regions

The input functions for training and testing are generated from GRFs, with a larger  $l$  leading to smoother functions (Fig. 1A). Hence, if the correlation length for training ( $l_{\text{train}}$ ) is different from the correlation length for testing ( $l_{\text{test}}$ ), then it is extrapolation (Ex.). The level of extrapolation can be represented by the difference between  $l_{\text{train}}$  and  $l_{\text{test}}$ .

For both problems, we choose  $l_{\text{train}}$  and  $l_{\text{test}}$  in the range  $\{0.1, 0.2, \dots, 1.0\}$ . The training dataset size is chosen as 1000. The  $L^2$  relative errors of DeepONets trained and tested on datasets with different  $l_{\text{train}}$  and  $l_{\text{test}}$  are shown in Figs. 1B and C. When  $l_{\text{train}} = l_{\text{test}}$  (Figs. 1B and C, diagonal dash lines), the training and test functions are sampled from the same space, and thus it is interpolation and the error is smaller than  $10^{-2}$ . In the bottom right region where  $l_{\text{train}} > l_{\text{test}}$ , the error of DeepONet is larger. When  $l_{\text{train}} < l_{\text{test}}$  (Figs. 1B and C, left top region), although it is extrapolation, DeepONet still has a small error, i.e., DeepONet can predict accurately for smoother functions. Therefore, we have three scenarios:

$$\text{Prediction} = \begin{cases} \text{Interpolation (In.)}, & \text{when } l_{\text{train}} = l_{\text{test}}, \\ \text{Ex.}^-, & \text{when } l_{\text{train}} < l_{\text{test}}, \\ \text{Ex.}^+, & \text{when } l_{\text{train}} > l_{\text{test}}. \end{cases}$$

Here, the extrapolation for functions with smaller  $l$  (i.e., less smooth) is denoted by  $\text{Ex.}^+$ , while extrapolation for functions with larger  $l$  (i.e., smoother) is denoted by  $\text{Ex.}^-$ .

### 2.4. Extrapolation complexity and 2-Wasserstein distance

We have tested DeepONet with different levels of extrapolation for  $\text{Ex.}^+$ . To quantify the extrapolation complexity, we measure the distance between two spaces of GRFs using the 2-Wasserstein ( $W_2$ ) metric [49], as was suggested in Ref. [8]. Let us consider two Gaussian processes  $\mathcal{GP}(m_1, k_1)$  and  $\mathcal{GP}(m_2, k_2)$  defined on a space  $X$ , where  $m_1, m_2$  are the mean functions and  $k_1, k_2$  are the covariance functions. A covariance function  $k_i$  is associated with a covariance operator  $K_i : L^2(X) \rightarrow L^2(X)$  given by

$$[K_i \phi](x) = \int_X k_i(x, s) \phi(s) ds, \quad \forall \phi \in L^2(X).$$

Then, the  $W_2$  metric between the two GPs is obtained as

$$W_2 := \left\{ \|m_1 - m_2\|_2^2 + \text{Tr} \left[ K_1 + K_2 - 2 \left( K_1^{\frac{1}{2}} K_2 K_1^{\frac{1}{2}} \right)^{\frac{1}{2}} \right] \right\}^{\frac{1}{2}},$$

where  $\text{Tr}$  is the trace. Following the definition, a larger  $W_2$  distance represents a greater difference between two GRF spaces.

We show the testing error between each pair of GRFs with different correlation lengths in Figs. 1B, C and D, and the relationship between the error and  $W_2$  distance is shown in Figs. 1E, F and G. We find that the extrapolation error grows with respect to the  $W_2$  distance between the two GRF spaces. Specifically, for the ODE problem, we have

$$\text{Error} \propto W_2^{1.7}. \quad (2)$$

For the diffusion-reaction equation with the coefficient  $k = 0.01$  and  $D = 0.01$ , we have

$$\text{Error} \propto W_2^{2.1}, \quad (3)$$

and for the coefficient  $k = 0.5$  and  $D = 0.01$ ,

$$\text{Error} \propto W_2^{1.9}. \quad (4)$$

Therefore, the  $W_2$  distance between the training and test spaces can be used as a measure of the extrapolation complexity. For the diffusion-reaction equation, we have observed similar behavior for three more cases (Fig. D.10): (1)  $k = 0.01$  and  $D = 1.0$ , (2) reaction term  $ku$  is linear with  $k = 0.01$  and  $D = 0.01$ , and (3) reaction term  $ku^3$  is cubic with  $k = 0.01$  and  $D = 0.01$ .

In Eqs. (2), (3), and (4), the test errors of different problems converge with different rates. For the diffusion-reaction equation, the value of  $k$  has an influence on the convergence rate. To further confirm if the influence of  $k$  on the convergence rate is significant, we compute the 95% confidence intervals for the two convergence rates above as  $[2.01, 2.25]$  and  $[1.79, 1.92]$ , and the  $p$ -values of the two-sided T-test is 0.0001, which implies the difference of the convergence rates is significant. Moreover, we observe that for a larger value of  $k$ , the extrapolation error grows slower, which is a surprising result since larger  $k$  leads to a stronger nonlinearity in the PDE. This is a preliminary result, and further investigation about this observation is required in future work.

## 2.5. Understanding the extrapolation error

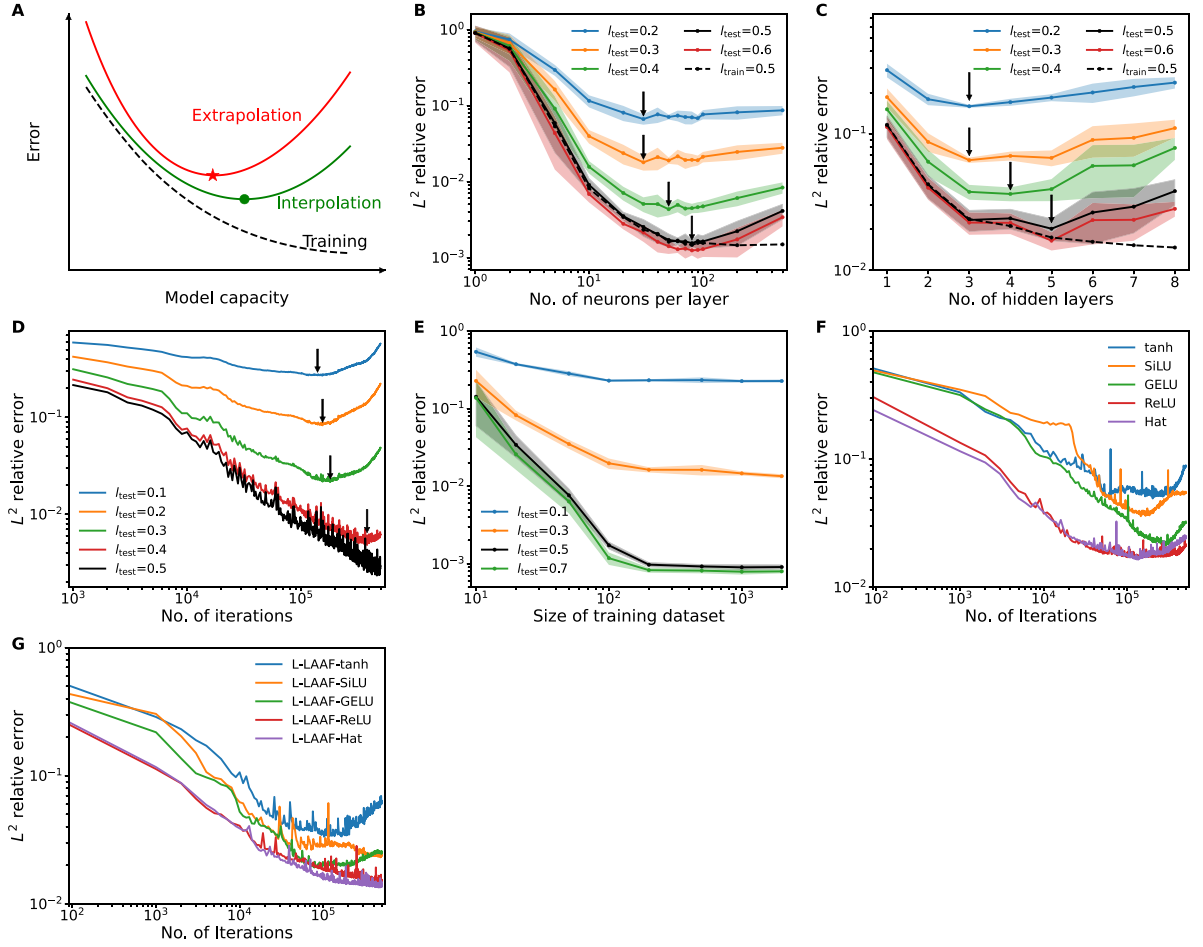
To further understand the extrapolation error, we use the diffusion-reaction equation as an example and investigate several factors that contribute to the extrapolation error. Unless otherwise stated, we use the following hyperparameters. The DeepONet has one hidden layer for the branch net and two hidden layers for the trunk net, each with 100 neurons per layer. The activation function for both branch and trunk nets is ReLU. The correlation length of the training dataset is  $l_{\text{train}} = 0.5$ . In this case,  $l_{\text{test}} < 0.5$ ,  $l_{\text{test}} > 0.5$  and  $l_{\text{test}} = 0.5$  represents  $\text{Ex}^+$ ,  $\text{Ex}^-$ , and interpolation, respectively. DeepONets are trained with the Adam optimizer with a learning rate of 0.001 for 500,000 iterations.

Our main findings of this subsection are as follows.

- Section 2.5.1: Similar to the classical U-shaped bias–variance trade-off curve for the test error of interpolation,  $\text{Ex}^+$  also has a U-shaped curve with respect to the network size and the number of training iterations. Compared with interpolation,  $\text{Ex}^+$  has larger error and earlier transition point.
- Section 2.5.2: Increasing the training dataset size results in better accuracy for  $\text{Ex}^+$ .
- Section 2.5.3: Different activation functions perform differently for  $\text{Ex}^+$ . The layer-wise locally adaptive activation functions (L-LAAF) [50,51] slightly outperform their corresponding non-adaptive activation functions.

### 2.5.1. Bias–variance trade-off for extrapolation

One of the central tenets in machine learning is the bias–variance trade-off [52,53], which implies the presence of a U-shaped curve for the test error in interpolation as the model capacity grows, while the training error decreases monotonically (Fig. 2A). The bottom of the U-shaped curve is achieved at the transition point which balances the bias and variance. To the left of the transition point, the model is underfitting, and to the right, it is overfitting. There are several factors that affect the model capacity, such as the network size and the number of training iterations. Here, we show that the test error of  $\text{Ex}^+$  also has a U-shaped curve.



**Fig. 2.** Test error of  $Ex.^+$  for the diffusion-reaction equation. (A) Schematic of typical curves for training error, interpolation test error, and  $Ex.^+$  test error versus model capacity. Test error has a U-shaped error curve. Red star and green dot represent the location where overfitting starts in  $Ex.^+$  and interpolation scenario, respectively. (B) The test error for different test datasets when using different network widths. The DeepONet contains 1 hidden layer for branch net and 2 hidden layers for trunk net. The arrows indicate the transition point between underfitting and overfitting. The curves and shaded regions represent the mean and one standard deviation of 10 runs. (C) The test error for different test datasets when using different number of hidden layers for both branch and trunk nets. The network width is 10. (D) The test error for different test datasets during the training of DeepONet. (E) The test error for different test datasets when using different training dataset sizes. (F) The test error when using different activation functions. (G) The test error when using layer-wise locally adaptive activation functions (L-LAAF). For all the experiments, we use  $l_{train} = 0.5$ .

To investigate the influence of the network size, we choose different network widths ranging from 1 to 500 (Fig. 2B). We use a smaller training dataset of 100 so that we can observe the overfitting more easily. The correlation length of the test datasets is chosen from 0.2 to 0.6. We find that the training loss always decreases, indicated by the black dashed line. For a fixed test dataset, we observe a U-shaped curve. Moreover, the transition point between underfitting and overfitting (indicated by the arrows in Fig. 2B) occurs earlier for  $Ex.^+$  than interpolation. Similar behavior is also observed when we fix the network width at 10 and increase the number of hidden layers of both branch and trunk nets (Fig. 2C).

Next, we study the two phases of underfitting and overfitting during the training of DeepONets. We use five test datasets generated with different correlation lengths  $l_{test}$  from 0.1 to 0.5. As shown in Fig. 2D, a smaller  $l_{test}$  leads to a larger extrapolation error during the entire training process. Moreover, the transition point between underfitting and overfitting (indicated by the arrows in Fig. 2D) occurs earlier for smaller  $l_{test}$ .

Here, we have observed the classical U-shaped curves for the test errors of interpolation and extrapolation. However, recently a “double-descent” test error curve has been observed for neural networks [54,55], i.e., when the model capacity is further increased, after a certain point the test error would decrease again. We did not observe the double-descent behavior for DeepONets, and one possible reason is that our model capacity is not large enough.

### 2.5.2. Training dataset size

Next, we explore the effect of training dataset size in both interpolation and extrapolation cases. The dataset size ranges from 10 to 2000, and we show the test errors of four datasets in Fig. 2E. It is well known that a larger training dataset leads to better accuracy for interpolation. As shown in Fig. 2E, for extrapolation, a larger training dataset still leads to a smaller test error. However, the improvement exhibits a diminishing trend as the dataset size increases.

### 2.5.3. Activation function

Another factor that affects the accuracy of DeepONet is the activation function. To evaluate the effect of activation functions, for the trunk nets, we consider four activation functions widely used in deep learning, including the hyperbolic tangent (tanh), rectified linear units (ReLU,  $\max(0, x)$ ) [56], sigmoid linear units (SiLU,  $\frac{x}{1+e^{-x}}$ ) [57], and Gaussian error linear units (GELU,  $x \cdot \frac{1}{2} \left[ 1 + \operatorname{erf}(x/\sqrt{2}) \right]$ ) [58]. In addition, we explore the Hat activation function [59] defined by

$$\text{Hat}(x) := \begin{cases} 0, & x < 0 \text{ or } x \geq 2, \\ x, & 0 \leq x < 1, \\ 2 - x, & 1 \leq x < 2. \end{cases}$$

For branch nets, we find that ReLU has similar or better performance than other activation functions, and thus we still use ReLU. The correlation lengths for training and testing datasets are  $l_{\text{train}} = 0.5$  and  $l_{\text{test}} = 0.3$ , respectively. ReLU and Hat exhibit similar results and outperform the rest of the activation functions (Fig. 2F). GELU also presents a satisfactory performance despite the overfitting after a certain number of training iterations.

Besides these commonly used activation functions, we also consider the layer-wise locally adaptive activation functions (L-LAAF) [50,51] in the form of  $\sigma(n \cdot a \cdot x)$ , where  $\sigma$  is a standard activation function,  $n$  is a scaling factor, and  $a$  is a trainable parameter. L-LAAF introduces the trainable parameter in each layer, thus leading to a local adaptation of activation function. The performance of L-LAAF depends on the scaling factor  $n$ , so we performed a grid search for  $n$  from 1, 2, 5, and 10 (Fig. E.11). With other settings remaining the same, L-LAAF with the optimal  $n$  slightly outperforms non-adaptive activation functions (Fig. 2G). L-LAAF-Hat and L-LAAF-ReLU perform the best among the five activation functions, and most importantly the tendency of overfitting is alleviated.

## 3. Reliable learning methods for safe extrapolation

As we demonstrate in Section 2, compared with interpolation, extrapolation leads to a much larger prediction error. However, extrapolation is usually unavoidable in real applications. In this section, we propose several reliable learning methods to guarantee a safe prediction in the extrapolation region.

### 3.1. Workflow of extrapolation

We continue to consider the setup of operator learning in Section 2.1. We assume that we have a training dataset  $\mathcal{T}$ , and then we train a DeepONet with  $\mathcal{T}$ . We denote this pre-trained DeepONet by  $\mathcal{G}_\theta$ , where  $\theta$  is the set of trainable parameters in the network. If the size of  $\mathcal{T}$  is large enough and the DeepONet is well trained, then the pre-trained DeepONet can have accurate predictions for interpolation. In this study, we do not develop new methods to improve the interpolation performance. Our goal is to have an accurate prediction for a new input function  $v$ , irrespective if  $v$  is inside or outside the distribution.

In general, it is very difficult to have an accurate prediction for extrapolation [29,30,60], e.g., see the simple problem in Fig. 1A and the examples of DeepONets in Refs. [8,10,16,17,31]. Hence, in order to reduce the extrapolation error, it is essential to have additional information. Here, we consider the following two scenarios and develop corresponding methods to address the extrapolation problem.

1. Physics (Section 3.3): We know the governing PDEs and/or the physical constraints of the system:

$$\mathcal{F}[u; v] = 0$$

with suitable initial and boundary conditions

$$\mathcal{B}[u; v] = 0.$$

2. Sparse observations (Sections 3.4 and 3.5): For a specific test input function  $v$ , we have sparse observations of the corresponding output function  $u = \mathcal{G}(v)$  at  $N_{\text{obs}}$  locations  $\{\xi_1, \xi_2, \dots, \xi_{N_{\text{obs}}}\}$ :

$$\mathcal{D} = \{(\xi_1, u(\xi_1)), (\xi_2, u(\xi_2)), \dots, (\xi_{N_{\text{obs}}}, u(\xi_{N_{\text{obs}}}))\}. \quad (5)$$

The availability of physics or new data varies in different scenarios. For example, if we forecast the weather in the next few days, we could utilize the data from the last few days. However, if we predict the climate in decades, the data will be unavailable, and we will have to resort to physics.

Since we aim to handle any input function  $v$ , the first step is to determine if  $v$  is inside or outside the distribution, which is discussed in Section 3.2. If it is interpolation, then it becomes trivial, i.e., we only need to predict the output by the trained DeepONet; otherwise, we need to extrapolate via the proposed methods. The entire workflow is shown in Fig. 3A.

### 3.2. Determination of interpolation or extrapolation

One approach to determine interpolation or extrapolation is to compare the new function  $v$  with the functions in the training dataset. If  $v$  is less smooth than the training functions, then it is extrapolation, otherwise it is interpolation as we presented in Section 2. We could also detect extrapolation through uncertainty quantification [26]. Here, because we have extra information of either physics or observations, we first predict the output  $\tilde{u}$  by the pre-trained DeepONet  $\mathcal{G}_\theta$ , i.e.,  $\tilde{u} = \mathcal{G}_\theta(v)$ , and then check if the  $\tilde{u}$  is consistent with the physics  $\mathcal{F}$  or observations  $\mathcal{D}$ .

Specifically, we propose to compute one of the following errors of mismatch:

- Mismatch error of physics, i.e., the mean PDE residual:

$$\mathcal{E}_{\text{phys}} = \frac{1}{\text{Area}(\Omega)} \int_{\Omega} |\mathcal{F}[\tilde{u}; v]| d\xi,$$

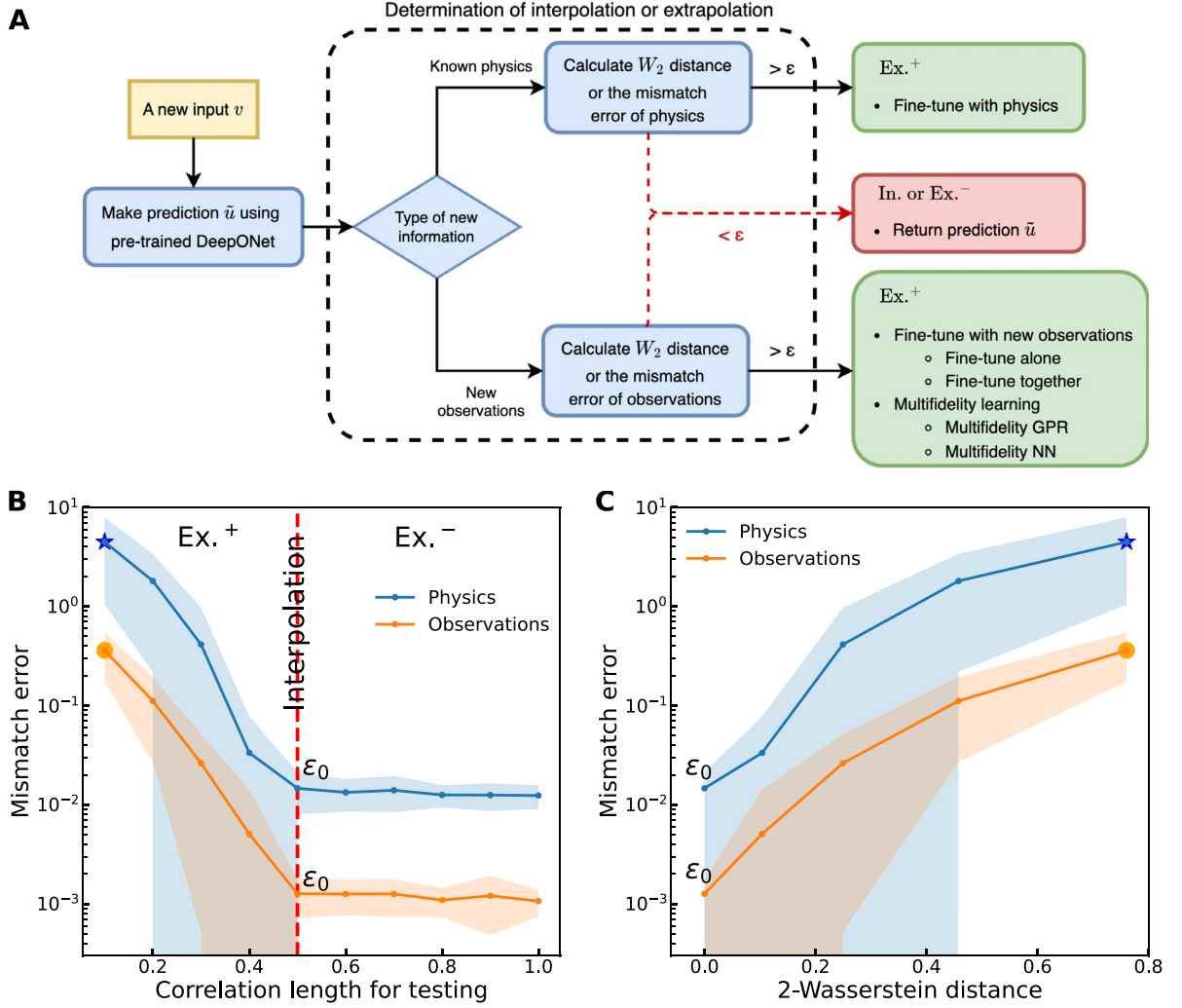
where  $\Omega$  is the domain of the PDE.

- Mismatch error of observations, i.e., the root relative squared error (RRSE):

$$\mathcal{E}_{\text{obs}} = \sqrt{\frac{\sum_{i=1}^{N_{\text{obs}}} (\tilde{u}(\xi_i) - u(\xi_i))^2}{\sum_{i=1}^{N_{\text{obs}}} u^2(\xi_i)}}.$$

To compute  $\mathcal{F}[\tilde{u}; v]$  in  $\mathcal{E}_{\text{phys}}$ , we need the derivatives of the DeepONet output  $\tilde{u}$  with respect to the trunk net input  $\xi$  while the branch net input is fixed at  $v$ . This can be computed via automatic differentiation as was done in physics-informed neural networks (PINNs) and physics-informed DeepONets [12,20].

To verify that  $\mathcal{E}_{\text{phys}}$  and  $\mathcal{E}_{\text{obs}}$  are good metrics of interpolation and extrapolation, we take the diffusion-reaction equation as an example. We train a DeepONet with the functions sampled from GRF with  $l_{\text{train}} = 0.5$ , and then compute  $\mathcal{E}_{\text{phys}}$  and  $\mathcal{E}_{\text{obs}}$  for different functions randomly sampled from GRFs with different correlation lengths  $l$ . We denote the value of  $\mathcal{E}_{\text{phys}}$  or  $\mathcal{E}_{\text{obs}}$  for  $l = l_{\text{train}}$  by  $\epsilon_0$ . In the interpolation and Ex.<sup>-</sup> regions, i.e.,  $l \geq l_{\text{train}}$ , the value of  $\mathcal{E}_{\text{phys}}$  or  $\mathcal{E}_{\text{obs}}$  is close to  $\epsilon_0$  (Fig. 3B). In the Ex.<sup>+</sup> region, i.e.,  $l < l_{\text{train}}$ , the value of  $\mathcal{E}_{\text{phys}}$  or  $\mathcal{E}_{\text{obs}}$  is much larger than  $\epsilon_0$ . Moreover, for two spaces with larger 2-Wasserstein distance, the mismatch error is also larger (Fig. 3C). Therefore, we can select the threshold  $\epsilon = \alpha \epsilon_0$ , where  $\alpha \geq 1$  is a user specified tolerance factor, and then compare  $\mathcal{E}_{\text{phys}}$  or  $\mathcal{E}_{\text{obs}}$  with  $\epsilon$ . If  $\mathcal{E}_{\text{phys}} > \epsilon$  or  $\mathcal{E}_{\text{obs}} > \epsilon$ , then it is Ex.<sup>+</sup>; otherwise, it is interpolation or Ex.<sup>-</sup> (Fig. 3A).



**Fig. 3.** Flowchart of predicting the output  $\tilde{u}$  for an input  $v$ . (A) For extrapolation, we use one of the following learning methods: fine-tune with physics, fine-tune with sparse new observations, or multifidelity learning. Here,  $\epsilon$  denotes a user specified threshold to determine interpolation or extrapolation. (B and C) Mismatch error of physics ( $\mathcal{E}_{\text{phys}}$ ) or observations ( $\mathcal{E}_{\text{obs}}$ ) for the diffusion-reaction equation. (B) Mismatch error for different testing correlation length. (C) Mismatch error for different 2-Wasserstein distance. The correlation length for training is 0.5. The number of new observations for  $\mathcal{E}_{\text{obs}}$  is 100. The curves and shaded regions represent the mean and one standard deviation of 100 test functions.

### 3.3. Extrapolation via fine-tuning with physics (FT-Phys)

We first discuss how to extrapolate with the additional information of physics (Algorithm 1). For a new input  $v$  in the extrapolation region, the prediction  $\tilde{u}$  by the pre-trained DeepONet  $\mathcal{G}_\theta$  may not satisfy the PDE. We propose to fine-tune  $\mathcal{G}_\theta$  to minimize the loss

$$\mathcal{L}_{\text{phys}} = w_{\mathcal{F}} \mathcal{L}_{\mathcal{F}} + w_{\mathcal{B}} \mathcal{L}_{\mathcal{B}}, \quad (6)$$

where  $\mathcal{L}_{\mathcal{F}}$  is the loss of PDE residuals

$$\mathcal{L}_{\mathcal{F}} = \frac{1}{|\mathcal{T}_{\mathcal{F}}|} \sum_{\xi \in \mathcal{T}_{\mathcal{F}}} |\mathcal{F}[\mathcal{G}_{\theta}(v)(\xi); v]|^2,$$

$\mathcal{L}_{\mathcal{B}}$  is the loss of initial and boundary conditions

$$\mathcal{L}_{\mathcal{B}} = \frac{1}{|\mathcal{T}_{\mathcal{B}}|} \sum_{\xi \in \mathcal{T}_{\mathcal{B}}} |\mathcal{B}[\mathcal{G}_{\theta}(v)(\xi); v]|^2,$$

and  $w_{\mathcal{F}}$  and  $w_{\mathcal{B}}$  are the weights.  $\mathcal{T}_{\mathcal{F}}$  and  $\mathcal{T}_{\mathcal{B}}$  are two sets of points sampled in the domain and on the initial and boundary locations, respectively. A DeepONet has two subnetworks (a branch net and a trunk net), and thus we could choose to fine-tune the entire DeepONet or only one subnetwork, while other parts remain unchanged. Moreover, as each subnetwork has multiple layers, we can also only fine-tune certain layers of the subnetwork. In this study, we consider five approaches: (1) fine-tuning both trunk and branch nets (Branch & Trunk), (2) fine-tuning the branch net, (3) fine-tuning the trunk net, (4) fine-tuning the last one layer of the trunk net (Trunk last one), and (5) fine-tuning the last two layers of the trunk net (Trunk last two).

---

**Algorithm 1: Extrapolation via fine-tuning with physics.**


---

**Input:** A pre-trained DeepONet  $\mathcal{G}_{\theta}$ , and a new input function  $v$

**Data:** Physics information  $\mathcal{F}$  and  $\mathcal{B}$

**Output:** Prediction  $u = \mathcal{G}_{\theta}(v)$

- 1 Fix the input of the branch net of  $\mathcal{G}_{\theta}$  to be  $v$ ;
  - 2 Sample the two sets of training points  $\mathcal{T}_{\mathcal{F}}$  and  $\mathcal{T}_{\mathcal{B}}$  for the equation and initial/boundary conditions;
  - 3 Select the weights  $w_{\mathcal{F}}$  and  $w_{\mathcal{B}}$ ;
  - 4 Train all the parameters or a subset of the parameters of  $\mathcal{G}_{\theta}$  by minimizing the loss  $\mathcal{L}_{\text{phys}}$  of Eq. (6);
- 

The idea of first pre-training and then fine-tuning is also relevant to the field of transfer learning (TL) [38–40], which aims to extract the knowledge from one or more source tasks and then apply the knowledge to a target task. However, most TL methods are developed to deal with covariate shift, label shift (or prior shift, target shift), and conditional shift, but not extrapolation.

As the branch net input is fixed at  $v$ , this fine-tuning method works in the same way as PINNs [4,5]. Compared with a PINN with random initialization, a pre-trained DeepONet provides a better initialization, which makes the training much faster, especially when the extrapolation complexity is small. The proposed method also uses the same technique as physics-informed DeepONets [12,20], but physics-informed DeepONets still require the target function space for training.

### 3.4. Extrapolation via fine-tuning with sparse new observations

With the information of new sparse observations  $\mathcal{D}$  in Eq. (5), we propose a few different approaches for Ex.<sup>+</sup> scenario. Similar to the fine-tuning with physics, the first proposed method is also in the spirit of transfer learning, i.e., we fine-tune the pre-trained DeepONet with the new observations (Algorithm 2).

*Fine-tune with new observations alone (FT-Obs-A).* As the pre-trained DeepONet is not consistent with the ground truth observations, we fine-tune the DeepONet to better fit the observations. Specifically, we further train the DeepONet by minimizing the mean squared error (MSE)

$$\mathcal{L}_{\text{obs}} = \frac{1}{N_{\text{obs}}} \sum_{i=1}^{N_{\text{obs}}} (\mathcal{G}_{\theta}(v)(\xi_i) - u(\xi_i))^2. \quad (7)$$

In this approach, we only use the new observations for fine-tuning, and thus we call it “fine-tune alone” to distinguish from the next approach.

The approach above is simple and has low computational cost. However, as  $N_{\text{obs}}$  is usually very small, it may have the issue of overfitting. It may also have the issue of catastrophic forgetting [61,62], i.e., DeepONet would forget previously learned information upon learning the new observations. Considering the fact that both the training dataset and the new observations satisfy the same operator  $\mathcal{G}$ , they can be learned by the same DeepONet at the same time. Hence, we propose the following fine-tuning approach to prevent overfitting and catastrophic forgetting.

**Algorithm 2: Extrapolation via fine-tuning with new observations.**

- 
- Input:** A pre-trained DeepONet  $\mathcal{G}_\theta$ , and a new input function  $v$   
**Data:** New observations  $\mathcal{D}$   
**Output:** Prediction  $u = \mathcal{G}_\theta(v)$
- 1 Fix the input of the branch net of  $\mathcal{G}_\theta$  to be  $v$ ;
  - 2 Fine-tune  $\mathcal{G}_\theta$  by minimizing the loss of Eq. (7) for fine-tuning alone or Eq. (8) for fine-tuning together;
- 

*Fine-tune with training data and new observations together (FT-Obs-T).* Instead of fitting DeepONet with only the new observations, we fine-tune the pre-trained DeepONet with new observed data together with the original training dataset  $\mathcal{T}$  via the loss

$$\mathcal{L}_{\mathcal{T},\text{obs}} = \mathcal{L}_{\mathcal{T}} + \lambda \mathcal{L}_{\text{obs}} \quad (8)$$

where  $\mathcal{L}_{\mathcal{T}}$  is the original loss for  $\mathcal{T}$ , e.g., MSE loss, and  $\lambda$  is a weight.

Compared with the FT-Obs-A above, FT-Obs-T keeps learning from  $\mathcal{T}$  via the loss of  $\mathcal{L}_{\mathcal{T}}$ , which mitigates the problem of catastrophic forgetting. Moreover,  $\mathcal{L}_{\mathcal{T}}$  has an effect of regularization to prevent the overfitting of new sparse observations. By tuning the value of  $\lambda$ , we can balance remembering the original information  $\mathcal{T}$  and learning from the new information  $\mathcal{D}$ .

### 3.5. Extrapolation via multifidelity learning with sparse new observations

In all the previous methods, we use the idea of fine-tuning. Here, we propose a different approach based on multifidelity learning [33]. The idea of multifidelity learning is that instead of learning from a large dataset of high accuracy (i.e., high fidelity), we only use a small high-fidelity dataset complemented by another dataset of low accuracy (i.e., low fidelity). Specifically, to predict  $\mathcal{G}(v)$  for the new function  $v$ , the sparse new observations  $\mathcal{D}$  is the high-fidelity dataset, while the prediction  $\tilde{u} = \mathcal{G}_\theta(v)$  from the pre-trained DeepONet is the low-fidelity dataset.

We integrate high- and low-fidelity datasets via two multifidelity methods: multifidelity Gaussian process regression (MF-GPR) [34] or multifidelity neural networks (MF-NN) [35–37]. In MF-GPR, we model the high- and low-fidelity functions by Gaussian processes with the radial-basis function kernel in Eq. (1). Then, the model is trained by minimizing the negative log marginal likelihood on the datasets. In MF-NN, we have one fully-connected neural network to learn the low-fidelity dataset and another fully-connected neural network to learn the correlation between low- and high-fidelity. For more details of MF-NN, see Refs. [35,36]. The algorithm is shown in Algorithm 3.

**Algorithm 3: Extrapolation via multifidelity learning with new observations.**

- 
- Input:** A pre-trained DeepONet  $\mathcal{G}_\theta$ , and a new input function  $v$   
**Data:** New observations  $\mathcal{D}$   
**Output:** Prediction  $u$
- 1 Compute the prediction  $\tilde{u} = \mathcal{G}_\theta(v)$ ;
  - 2 Sample a set of dense points  $\mathcal{S} = \{\xi_i\}_{i=1}^{|\mathcal{S}|}$  and use  $\{(\xi_i, \tilde{u}(\xi_i))\}_{i=1}^{|\mathcal{S}|}$  as the low-fidelity dataset;
  - 3 Use  $\mathcal{D}$  as the high-fidelity dataset;
  - 4 Train a multifidelity model (MF-GPR or MF-NN) on the multifidelity datasets;
- 

## 4. Extrapolation results

In this section, we test the proposed extrapolation methods with different PDEs. The hyperparameters used in this study can be found in Appendix B. For all experiments, the Python library DeepXDE [5] is utilized to implement the algorithms and train the neural networks. The code in this study is publicly available from the GitHub repository <https://github.com/lu-group/deeponet-extrapolation>.

To demonstrate the effectiveness of the extrapolation methods, we consider three different baselines. We use the pre-trained DeepONet as the first baseline to show the Ex.<sup>+</sup> error without any additional information. Physics-informed DeepONet (PI-DeepONet) [63] is taken as another baseline model, which is trained by the PDE loss and

**Table 1**

**$L^2$  relative error of different methods for various benchmarks.** The table consists of three parts: the first part is the interpolation and Ex.<sup>+</sup> errors of DeepONet and PI-DeepONet as the reference, and the second part is comprised of methods with the additional information of physics, while the third part contains approaches with sparse new observations. The errors represent the mean and one standard deviation of 100 test cases. For each case, bold font indicates the smallest two errors, and the underlined text indicates the smallest error.

|                              | Antiderivative<br>Section C | Diffusion-reaction<br>Section 4.1 | Burgers'<br>Section 4.2 | Advection<br>Section 4.3 | Poisson<br>Section 4.4 | Lid-driven cavity flow<br>Section 4.5 |                     |
|------------------------------|-----------------------------|-----------------------------------|-------------------------|--------------------------|------------------------|---------------------------------------|---------------------|
|                              |                             |                                   |                         |                          |                        | $u$                                   | $v$                 |
| DeepONet In.                 | 0.93 ± 0.20%                | 0.74 ± 0.29%                      | 2.21 ± 1.11%            | 1.30 ± 0.26%             | 0.09 ± 0.04%           | 1.51%                                 | 1.02%               |
| DeepONet Ex. <sup>+</sup>    | 11.6 ± 5.26%                | 10.4 ± 6.24%                      | 6.53 ± 3.33%            | 8.75 ± 6.42%             | 14.8 ± 12.0%           | 12.8%                                 | 12.4%               |
| PI-DeepONet In.              | 0.81 ± 0.69%                | 0.42 ± 0.20%                      | 4.96 ± 1.80%            | 3.60 ± 0.70%             | –                      | –                                     | –                   |
| PI-DeepONet Ex. <sup>+</sup> | 17.4 ± 6.01%                | 10.2 ± 6.30%                      | 9.27 ± 3.93%            | 10.4 ± 4.48%             | –                      | –                                     | –                   |
| PINN                         | 1.77 ± 2.14%                | 0.49 ± 0.24%                      | 1.34 ± 0.59%            | 1.67 ± 0.53%             | –                      | –                                     | –                   |
| FT-Phys w/<br>Branch & Trunk | <b>1.84 ± 1.49%</b>         | 0.37 ± 0.16%                      | <b>0.74 ± 0.36%</b>     | 1.05 ± 0.30%             | –                      | –                                     | –                   |
| FT-Phys w/<br>Branch         | 5.09 ± 3.66%                | 1.28 ± 0.62%                      | 6.82 ± 3.24%            | 2.65 ± 1.17%             | –                      | –                                     | –                   |
| FT-Phys w/<br>Trunk          | <b>1.52 ± 1.00%</b>         | <b>0.32 ± 0.15%</b>               | <b>0.68 ± 0.39%</b>     | <b>0.93 ± 0.23%</b>      | –                      | –                                     | –                   |
| FT-Phys w/<br>Trunk last one | 2.29 ± 1.65%                | 0.48 ± 0.21%                      | 0.94 ± 0.54%            | 1.01 ± 0.31%             | –                      | –                                     | –                   |
| FT-Phys w/<br>Trunk last two | –                           | <b>0.28 ± 0.14%</b>               | 0.76 ± 0.37%            | <b>0.99 ± 0.26%</b>      | –                      | –                                     | –                   |
| No. of observations          | 5 points                    | 50 points                         | 100 points              | 100 points               | 50 points              | 100 points                            | 100 points          |
| GPR                          | 8.63 ± 14.1%                | 9.63 ± 4.55%                      | 18.6 ± 10.3%            | 16.7 ± 3.99%             | 12.1 ± 6.57%           | 59.2 ± 14.9%                          | 60.5 ± 15.4%        |
| FT-Obs-A                     | 5.98 ± 4.15%                | <b>3.36 ± 1.72%</b>               | <b>4.00 ± 1.99%</b>     | <b>3.91 ± 1.52%</b>      | <b>4.15 ± 2.89%</b>    | <b>4.90 ± 0.85%</b>                   | <b>5.07 ± 0.81%</b> |
| FT-Obs-T                     | 5.61 ± 4.19%                | <b>2.69 ± 1.51%</b>               | <b>3.44 ± 1.72%</b>     | <b>3.16 ± 1.34%</b>      | <b>3.01 ± 2.11%</b>    | <b>2.23 ± 0.28%</b>                   | <b>2.43 ± 0.33%</b> |
| MF-GPR                       | <b>5.17 ± 3.74%</b>         | 7.96 ± 3.45%                      | 5.15 ± 2.97%            | 5.47 ± 2.87%             | 5.25 ± 3.38%           | 26.4 ± 2.56%                          | 21.0 ± 2.30%        |
| MF-NN                        | <b>4.79 ± 3.02%</b>         | 4.50 ± 2.20%                      | 5.15 ± 2.56%            | 6.90 ± 6.47%             | 7.63 ± 5.16%           | 12.4 ± 1.26%                          | 14.6 ± 2.72%        |

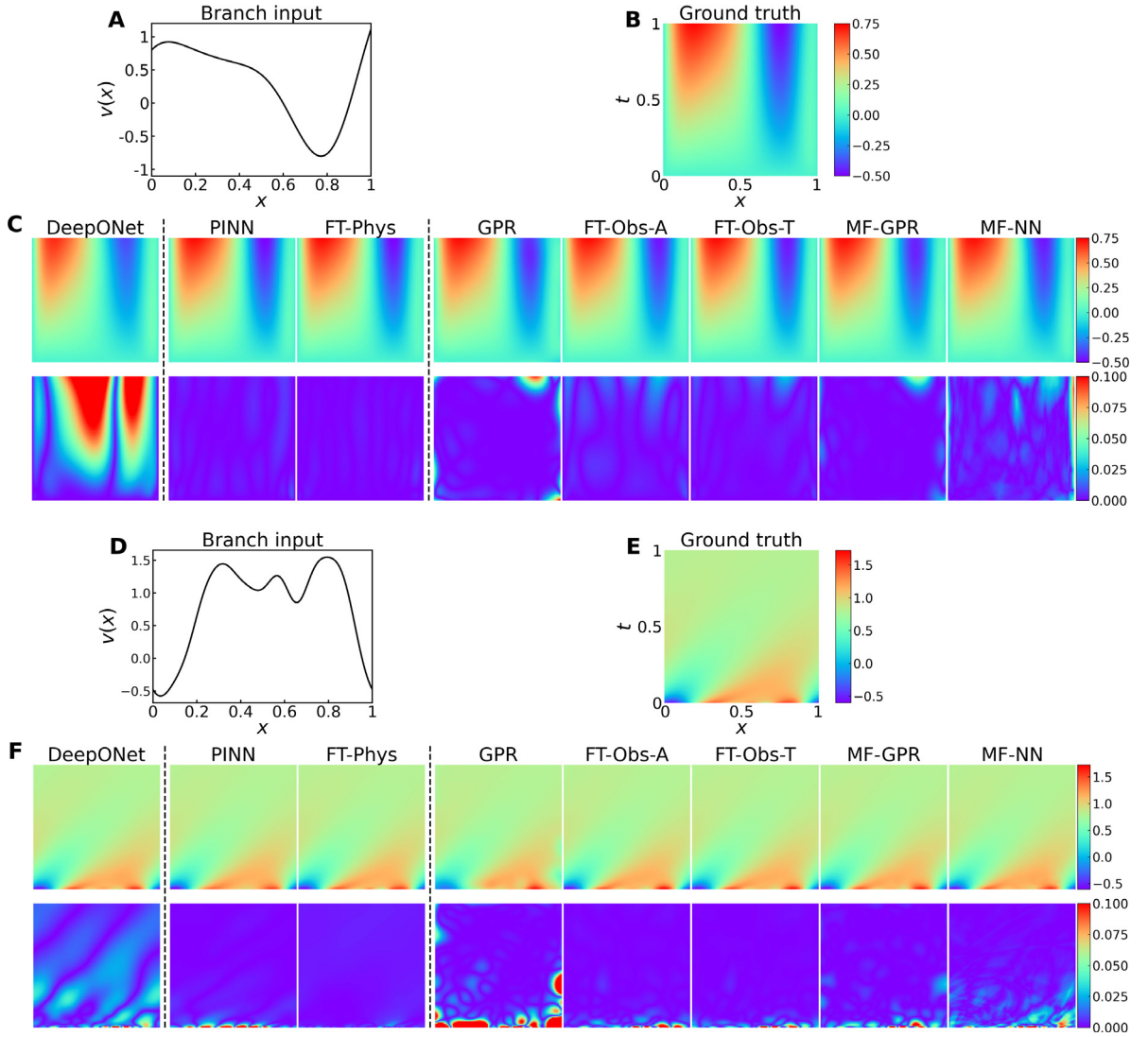
initial/boundary conditions. When we know the governing physics, we also use PINN as the baseline for the method of fine-tuning with physics. The network size and activation function of PINN are the same as those of trunk net of DeepONets. When we have extra new observations, we perform a Gaussian process regression (GPR) with the observations as a strong single-fidelity baseline for fine-tuning with observations and multifidelity learning. The new observations are randomly sampled in the domain unless otherwise stated. Here, we first present a summary of the accuracy of all the methods for various benchmarks in Table 1, and more results for each problem are provided in Appendix D. We also provide the example of antiderivative in Appendix C.

#### 4.1. Diffusion-reaction equation

Next, we consider the diffusion-reaction equation in Section 2.2. We aim to train a DeepONet to learn the operator mapping from the source term  $v(x)$  to the solution  $u(x, t)$ . The source term  $v(x)$  is randomly sampled from a GRF with an RBF kernel with the correlation length  $l_{\text{train}} = 0.5$ . To test the Ex.<sup>+</sup>, we generate a test dataset of 100 functions with  $l_{\text{test}} = 0.2$ .

Tables 1 and D.9 summarize the  $L^2$  relative errors of different methods. The FT-Phys has the lowest error, and the FT-Obs-T works the best among methods that require new observations. The pre-trained DeepONet has an average  $L^2$  relative error of 10.4%, and PI-DeepONet has an average  $L^2$  relative error of 10.2% for the test dataset. When we have physics, fine-tuning with trunk net at a learning rate of 0.002 can achieve low  $L^2$  relative errors (0.32%) and significantly outperform the pre-trained DeepONet and GPR. As shown in Table D.9, when we have 20, 50, and 100 new observations, FT-Obs-A and FT-Obs-T generally outperform MF-GPR and MF-NN. However, when we have 200 new observations, GPR and MF-GPR reach low  $L^2$  relative errors because the diffusion-reaction equation is relatively simple with a smooth solution, and 200 points are sufficient for MF-GPR and GPR to obtain accurate results.

Figs. 4A, B and C are an example illustrating the predictions and absolute errors of all methods when we have 100 observations. We find that the locations of significant errors are similar between GPR and MF-GPR (Fig. 4C)



**Fig. 4.** Examples of extrapolation for the diffusion-reaction equation in Section 4.1 and the Burgers' equation in Section 4.2. (A to C) The diffusion-reaction equation. (A) A test input function. (B) The corresponding PDE solution. (C) Predictions (first row) and errors (second row) of different methods. FT-Phys and FT-Obs-T obtain the best results. (D to F) The Burgers' equation. (D) A test input function. (E) The corresponding PDE solution. (F) Predictions (first row) and errors (second row) of different methods. FT-Phys and FT-Obs-T obtain the best results.

due to the similarity of these two methods. FT-Obs-A and FT-Obs-T also have a similar profile of errors. It is worth noting that both PINN and FT-Phys have very low errors, so we cannot find a similar error between PINN and FT-Phys in this example, but it can be found in the advection equation in Section 4.3.

**Detailed results of FT-Phys.** We consider physics as additional information and hence we develop the approach of fine-tuning with physics. Seven different learning rates (i.e., 0.01, 0.005, 0.002, 0.001, 0.0005, 0.0002, and 0.0001)

are used to fine-tune the pre-trained DeepONet. Figs. F.12A–G display the  $L^2$  relative errors of 100 test functions for different learning rates. When the number of iterations is small, PINNs give larger  $L^2$  relative errors ( $> 100\%$ ), while the FT-Phys produces much lower errors ( $\sim 10\%$ ). This is because the parameters of PINNs are randomly initialized. By contrast, the initial parameters of the FT-Phys are given by the pre-trained DeepONet. For fine-tuning different parts of DeepONet, we select the best accuracy among different learning rates and summarize the results in Fig. F.12H. Fig. F.12I shows the  $L^2$  relative errors with respect to the learning rate for different approaches. The performance of PINN is susceptible to the change in learning rates (Fig. F.12I, black line). Fine-tuning of branch net ( $\sim 1\%$ ), in general, performs worse than other approaches regardless of learning rates. Fine-tuning of the entire DeepONet (branch & trunk), of the entire trunk net, or of the last two layers of trunk net performs the best and reaches a very low error for any learning rate between 0.01 and 0.0001. Fine-tuning of the last one layer of the trunk net also achieves good accuracy. The best accuracy of different FT-Phys approaches is shown in Table D.9.

*Effect of  $\lambda$  on FT-Obs-T.* We aim to find an optimal  $\lambda$  to achieve the lowest test errors. Also, we determine the effect of  $\lambda$  on test errors when using different numbers of observed points and different  $l_{\text{test}}$ . The results of 12 cases with different new observation numbers and testing correlation lengths are shown in Fig. G.15. We can see that the lowest error is obtained when  $\lambda$  is  $\sim 0.3$ , so we choose  $\lambda = 0.3$  as the default value in our experiments.

#### 4.2. Burgers' equation

Next, we consider the Burgers' equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [0, 1], \quad t \in [0, 1],$$

with a periodic boundary condition and an initial condition  $u_0(x) = v(x)$ . In this study,  $\nu$  is set at 0.1. Our goal is to train a DeepONet to learn the operator mapping from initial condition  $v(x)$  to the solution  $u(x, t)$ . The periodic function  $v(x)$  is sampled from a GRF with an exponential sine squared kernel (or periodic kernel) given by

$$k_l(x_1, x_2) = \exp\left(-\frac{2 \sin^2(\pi \|x_1 - x_2\|/p)}{l^2}\right),$$

where  $l$  is the correlation length of the kernel, and  $p$  is the periodicity of the kernel, which is chosen at 1. The correlation length for training is  $l_{\text{train}} = 1.0$ , and to test the Ex.<sup>+</sup>, we generate a test dataset of 100 functions with  $l_{\text{test}} = 0.6$ .

Tables 1 and D.10 summarize the  $L^2$  relative errors of different methods. The pre-trained DeepONet has an average  $L^2$  relative error of 6.53%, and PI-DeepONet has an average  $L_2$  relative error of 9.27% for the test dataset. When we have physics, fine-tuning of the trunk net at a learning rate of 0.002, or of the branch & trunk net at a learning rate of 0.001 can achieve low  $L^2$  relative errors (1.42%) and significantly outperforms the pre-trained DeepONet. As shown in Table D.10, when we have new observations, unlike the results of the diffusion-reaction equation, FT-Obs-A and FT-Obs-T consistently outperform MF-GPR and MF-NN, while FT-Obs-T works the best among methods that use new observations. Figs. 4D, E and F are an example of illustrating the prediction and absolute errors of all methods when we have 200 observations. In this example, pre-trained DeepONet has relatively large errors distributed in the whole domain. In contrast, the large errors of the remaining methods are gathered in the initial area of the domain.

*Detailed results of FT-Phys.* Here, we consider the approach of fine-tuning with physics. Seven different learning rates (i.e., 0.01, 0.005, 0.002, 0.001, 0.0005, 0.0002, and 0.0001) are used to fine-tune the pre-trained DeepONet. Figs. F.13A–G display the  $L^2$  relative errors of 100 test functions for different learning rates. When the number of iterations is small, PINNs give larger  $L^2$  relative errors ( $\sim 100\%$ ), while the FT-Phys produces much lower errors ( $\sim 7\%$ ), which is consistent with the results of the diffusion-reaction equation. For fine-tuning different parts of DeepONet, we select the best accuracy among different learning rates and summarize the results in Fig. F.13H. Fig. F.13I shows the  $L^2$  relative errors with respect to the learning rate for different approaches. The performance of PINN is very sensitive to the change in learning rates (Fig. F.13I, black line). When the learning rate is low ( $\sim 0.0001$ ), the  $L^2$  relative error of PINN is large ( $\sim 20\%$ ), while PINN can achieve a small  $L^2$  relative error ( $\sim 1\%$ ) when the learning rate is 0.002. Fine-tuning of the branch net ( $\sim 7\%$ ) performs worse than other approaches

regardless of learning rates. Fine-tuning of the entire DeepONet (branch & trunk), of the entire trunk net, of the last two trunk layers, or of the last one trunk layer performs similarly and reaches a low error (<1%) for any learning rate between 0.01 and 0.0001.

#### 4.3. Advection equation

Next, we consider the advection equation

$$\frac{\partial u}{\partial t} + v(x) \frac{\partial u}{\partial x} = 0, \quad x \in [0, 1], \quad t \in [0, 1],$$

with the initial condition  $u(x, 0) = \sin(\pi x)$  and boundary condition  $u(0, t) = \sin(\pi t/2)$ . To make sure that the advection velocity  $v(x)$  is larger than 0, we reformulate it in the form of  $v(x) = V(x) - \min_x V(x) + 1$ . We train a DeepONet to learn the operator mapping from  $v(x)$  to the solution  $u(x, t)$ .  $V(x)$  is sampled from a GRF with an RBF kernel with the correlation length  $l_{\text{train}} = 0.5$ . To test the Ex.<sup>+</sup>, we generate a test dataset of 100 functions with  $l_{\text{test}} = 0.2$ .

Tables 1 and D.11 summarize the  $L^2$  relative errors of different methods. The pre-trained DeepONet has an average  $L^2$  relative error of 8.75%, and PI-DeepONet has an average  $L_2$  relative error of 11.3% for the test dataset. When we have physics, fine-tuning of the trunk net achieves low  $L^2$  relative errors (0.93%). As shown in Table D.11, when we have new observations, like the results of Burgers' equation, FT-Obs-A and FT-Obs-T always outperform MF-GPR and MF-NN, and FT-Obs-T works the best. Fig. 5A, B and C are an example illustrating the prediction and absolute errors of all methods when we have 200 observations. In this example, PINN and FT-Phys have similar error profiles, and FT-Phys is more accurate than PINN, since the pre-trained DeepONet gives better initial parameters of the FT-Phys. Similar error profiles are also observed between FT-Obs-A and FT-Obs-T.

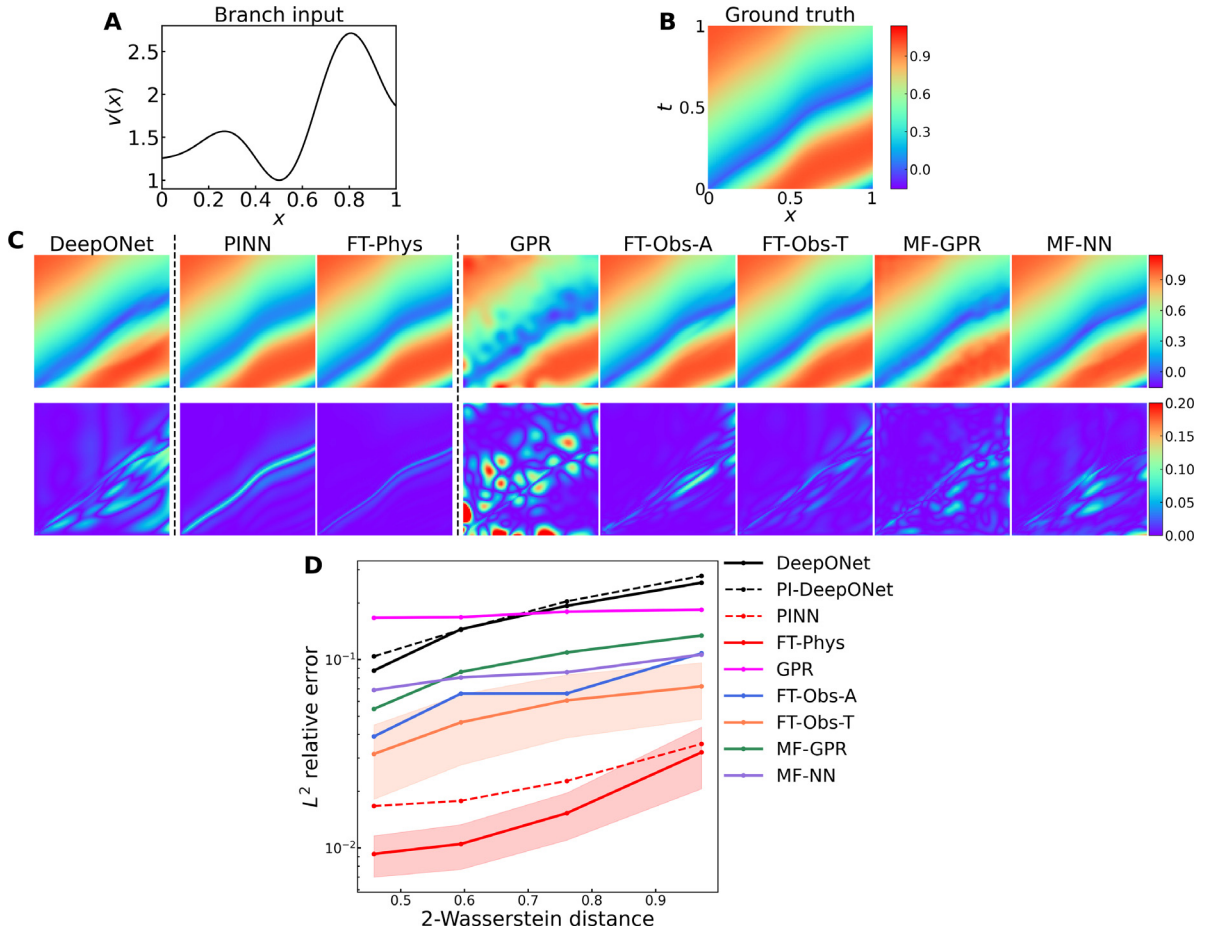
Moreover, we consider different Ex.<sup>+</sup> levels by using different correlation lengths for testing (Fig. 5D). Test datasets of 100 functions are generated from GFR with  $l_{\text{test}} = 0.2, 0.15, 0.1, \text{ and } 0.05$ . The 2-Wasserstein distances between the training space ( $l_{\text{train}} = 0.5$ ) and testing space are 0.4578, 0.5945, 0.7606, and 0.9721, respectively. We use 100 data points for fine-tuning methods. Decreasing the correlation lengths represents an increasing trend of extrapolation and results in a larger error, which also validates the result in Section 2.5.1. All proposed methods outperform the baseline models, DeepONet and GPR. When we have physics, FT-Phys outperforms PINN in all scenarios, and even under the great extrapolation with  $l_{\text{test}} = 0.05$ , it reaches a satisfactory error as low as 3.22%. With 100 measurements, FT-Obs-T achieves better performance compared with other fine-tune methods. For multifidelity methods, MF-NN performs better as the level of extrapolation increases, but both are slightly worse than FT-Obs methods.

*Detailed results of FT-Phys.* Now we consider the approach of fine-tuning with physics. Seven different learning rates (i.e., 0.01, 0.005, 0.002, 0.001, 0.0005, 0.0002, and 0.0001) are used to fine-tune the pre-trained DeepONet. Figs. F.14A–G display the  $L^2$  relative errors of 100 test functions for different learning rates. When the number of iterations is small, PINNs give larger  $L^2$  relative errors (>100%). In comparison, the FT-Phys produces much lower errors (~10%), which is consistent with the results of the diffusion-reaction equation and Burgers' equation. For fine-tuning different parts of DeepONet, we select the best accuracy among different learning rates and summarize the results in Fig. F.14H. Fig. F.14I shows the  $L^2$  relative errors with respect to learning rate for different approaches. The performance of PINN is susceptible to the change in learning rates (Fig. F.14I, black line). When the learning rate is low (~0.0001), the  $L^2$  relative error of PINN is large (~10%), while PINN can achieve a small  $L^2$  relative error (~2%) when the learning rate is 0.005. Fine-tuning of the branch net (~3%) performs worse than other approaches regardless of learning rates. Fine-tuning of the entire DeepONet (branch & trunk), of the entire trunk net, of the last two trunk layers, or of the last one trunk layer performs similarly and reaches a low error (~1%) at any learning rate between 0.01 and 0.0001.

#### 4.4. Poisson equation in a triangular domain with notch

To evaluate the performance of our methods in an unstructured domain, we consider the Poisson equation in a triangular domain with a notch

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + 10 = 0, \quad (x, y) \in \Omega,$$

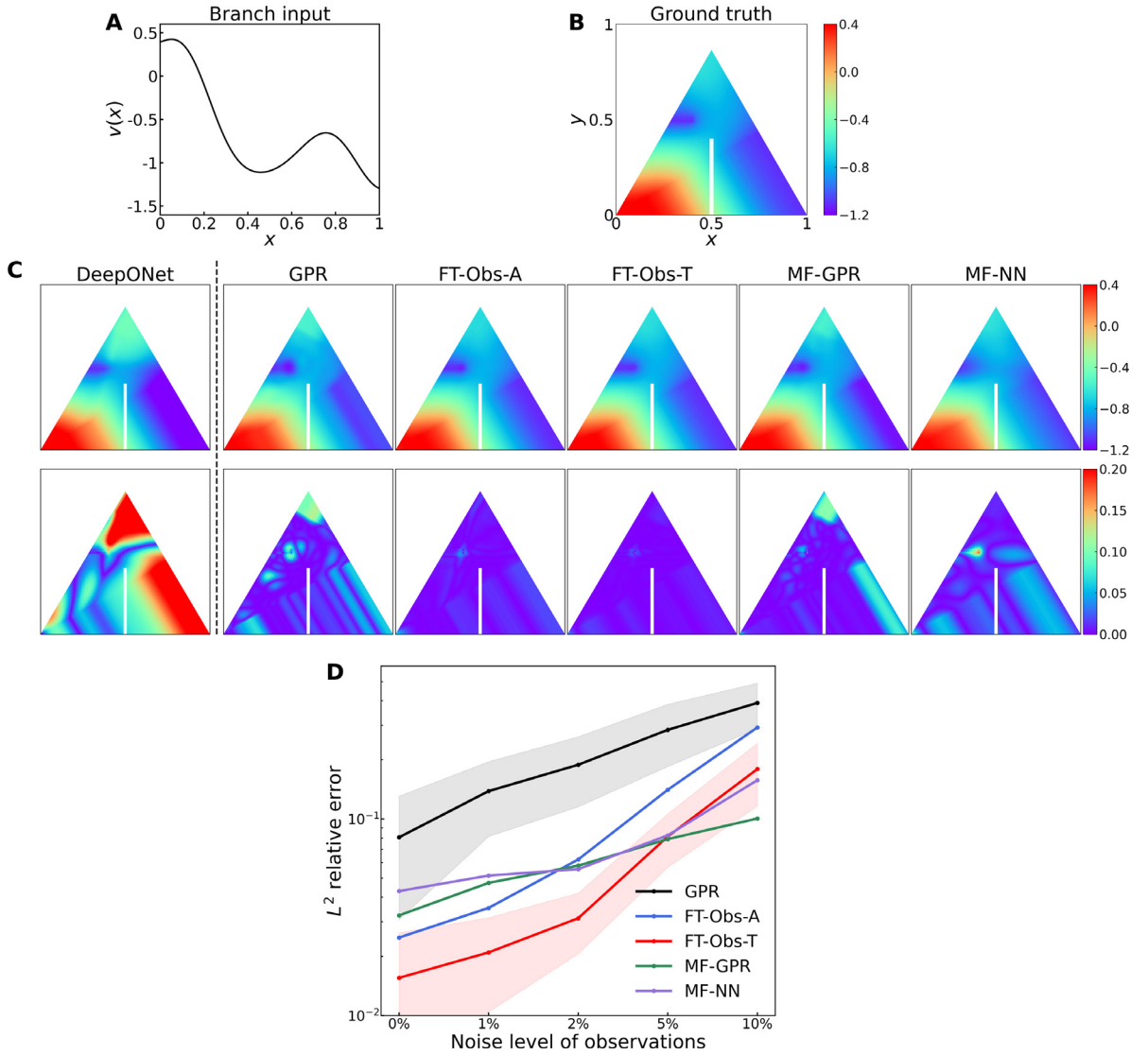


**Fig. 5.** Extrapolation for the advection equation in Section 4.3. (A) A test input function. (B) The corresponding PDE solution. (C) Predictions (first row) and errors (second row) of different methods. FT-Phys and FT-Obs-T obtain the best results. (D)  $L^2$  relative errors for different testing correlation lengths and 2-Wasserstein distances between the training and testing spaces.

where  $\Omega$  is a concave heptagon whose vertices are  $[0, 0]$ ,  $[0.5, 0.866]$ ,  $[1, 0]$ ,  $[0.51, 0]$ ,  $[0.51, 0.4]$ ,  $[0.49, 0.4]$ ,  $[0.49, 0]$ . The boundary condition  $u(x, y)|_{\partial\Omega} = v(x)$ , is only a function of its  $x$  coordinate. We train a DeepONet to learn the operator mapping from boundary condition  $v(x)$  to the solution  $u(x, t)$ .  $v(x)$  is sampled from a GRF with an RBF kernel with the correlation length  $l_{\text{train}} = 0.5$ . To test the Ex.<sup>+</sup>, we generate a test dataset of 100 functions with  $l_{\text{test}} = 0.2$ . The reference solution is obtained by the PDEtoolbox in Matlab with an unstructured mesh of 5082 nodes.

Tables 1 and D.12 summarize the  $L^2$  relative errors of different methods. The pre-trained DeepONet has an average  $L^2$  relative error of 14.8% for the test dataset. In this problem, we only tested the case of new observations, as FT-Phys and PINN failed to achieve a good accuracy due to the complex domain geometry with singularity points. Like the results of Burgers' equation and advection equation, FT-Obs-A and FT-Obs-T always outperform MF-GPR and MF-NN. The FT-Obs-T has the lowest errors (0.95% with 200 observations) among all the methods.

Figs. 6A, B and C are an example of illustrating the prediction and absolute errors of all methods when we have 200 observations. Similar to the results in Section 4.1, GPR and MF-GPR have similar error patterns, and MF-GPR is more accurate than GPR due to the additional low-fidelity dataset. FT-Obs-T and FT-Obs-A outperform other methods, and FT-Obs-T is slightly better than FT-Obs-A.



**Fig. 6.** Extrapolation for the Poisson equation in a triangular domain with a notch in Section 4.4. (A) A test input function. (B) The corresponding PDE solution. (C) Predictions (first row) and errors (second row) of different methods. FT-Obs-T obtains the best result. (D)  $L^2$  relative error for observations with different noise levels.

To validate the robustness of proposed fine-tuning methods, we test the performance under different levels of noise in the observations (Fig. 6D). With other settings remaining unchanged, increasing noise levels lead to a corresponding increase in  $L^2$  relative error. All fine-tuning methods exhibit better performance than GPR. When the noise level is less than 5%, FT-Obs-T reaches the best accuracy and as the noise gradually increases to 10%, MF-GPR method outperforms all other methods. Multifidelity methods are relatively insensitive to noise compared with FT-Obs.

For FT-Obs methods in the aforementioned experiments, we only fine-tune the entire DeepONet (both trunk and branch nets). In Table 2, we consider the five fine-tuning approaches proposed in FT-Phys. We find that for both FT-Obs-A and FT-Obs-T, fine-tuning of both the trunk and branch nets performs the best. Nevertheless, the variations observed among these five approaches do not exhibit considerable significance.

**Table 2**

$L^2$  relative error of FT-Obs methods for fine-tuning different DeepONet components for the Poisson equation in Section 4.4.  $l_{\text{train}} = 0.5$  and  $l_{\text{test}} = 0.2$ . The number of sparse observations is 50. Bold font indicates the smallest error in each row.

|          | Branch & Trunk                      | Branch            | Trunk             | Trunk last one    | Trunk last two    |
|----------|-------------------------------------|-------------------|-------------------|-------------------|-------------------|
| FT-Obs-A | <b><math>4.15 \pm 2.89\%</math></b> | $5.33 \pm 3.52\%$ | $4.65 \pm 2.77\%$ | $4.66 \pm 2.74\%$ | $4.55 \pm 2.70\%$ |
| FT-Obs-T | <b><math>3.01 \pm 2.11\%</math></b> | $3.33 \pm 2.18\%$ | $3.95 \pm 2.67\%$ | $3.93 \pm 2.56\%$ | $4.01 \pm 2.73\%$ |

#### 4.5. Lid-driven cavity flow in complex geometries

We evaluate the performance in the lid-driven cavity flow problem, which is a benchmark problem for viscous incompressible fluid flow. The incompressible flow is described by the Navier–Stokes equations,

$$\begin{aligned} \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} &= \nu \nabla^2 \mathbf{u} - \nabla p, \\ \nabla \cdot \mathbf{u} &= 0, \end{aligned}$$

where  $\mathbf{u}(\mathbf{x}, t) = (u, v)$  is the velocity field,  $\nu$  is the kinematic viscosity, and  $p(\mathbf{x}, t)$  is the pressure. The Reynolds number (Re) is chosen to be 1000. We consider different geometries by starting with a square with side length  $l = 1$  and gradually lifting the left bottom point with other three points fixed, i.e., the bottom line is described by  $l(x) = -mx + m$  with  $0 \leq m \leq 0.5$ . The top wall has a unit velocity in the  $x$ -direction. There is a flow circulation in the cavity, and increasing value of  $m$  represents more extreme cases, see two examples in Figs. 7A and B.

The goal is to learn the operator mapping from the boundary line  $l(x)$  to the velocity  $\mathbf{u}(\mathbf{x}, t)$ . We take  $m = 0, 0.02, 0.04, \dots, 0.4$  for generating the training dataset and select  $m = 0.41, 0.42, 0.43, \dots, 0.5$  for extrapolation testing. Larger  $m$  represents more aggressive extrapolation. The reference solution is obtained by the finite element method with a nonuniform mesh of 10201 nodes. For fine-tuning with new observations and multifidelity methods, 100 points are randomly chosen as the new information. To avoid the randomness, this process is repeated for 10 times.

For the prediction of the horizontal velocity  $u$  (Fig. 7C), the baseline model GPR has the largest  $L^2$  relative error. Multifidelity methods outperform the single-fidelity GPR, and MF-NN has a better accuracy than MF-GPR (Fig. 7C), yet both are still far from being satisfactory. We note that in previous examples, the RBF kernel works well for MF-GPR, but in this problem, MF-GPR with the RBF kernel has a large error, while the Matern kernel with  $\nu = 1.5$  performs better (Fig. G.16). Fine-tuning with observations provides a considerable improvement on the accuracy and reduces the  $L_2$  relative error to  $10^{-2}$ . FT-Obs-T works the best among all proposed method. The prediction of the vertical velocity  $v$  (Fig. 7D) also has a similar behavior.

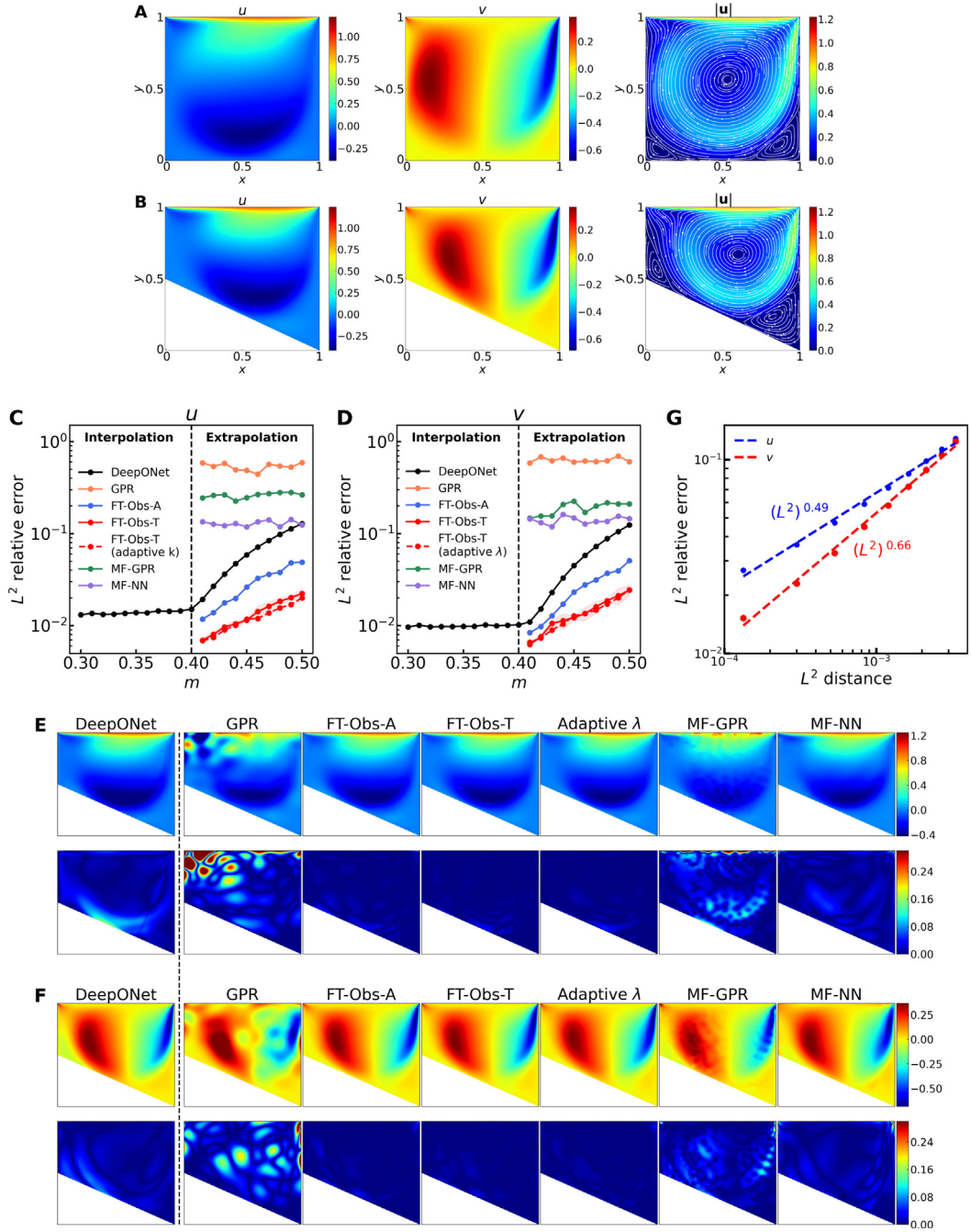
In FT-Obs-T, besides taking the weight  $\lambda$  as constant value, we also consider an approach to adaptively update the value of  $\lambda$  during training. Specifically, we use  $k = 0.1$  as the initial value, and then after every 100 iterations,  $\lambda$  is updated by gradient descent, i.e.,

$$\lambda \leftarrow \lambda + \gamma_\lambda \frac{\partial \mathcal{L}_{\mathcal{T}, \text{obs}}}{\partial \lambda},$$

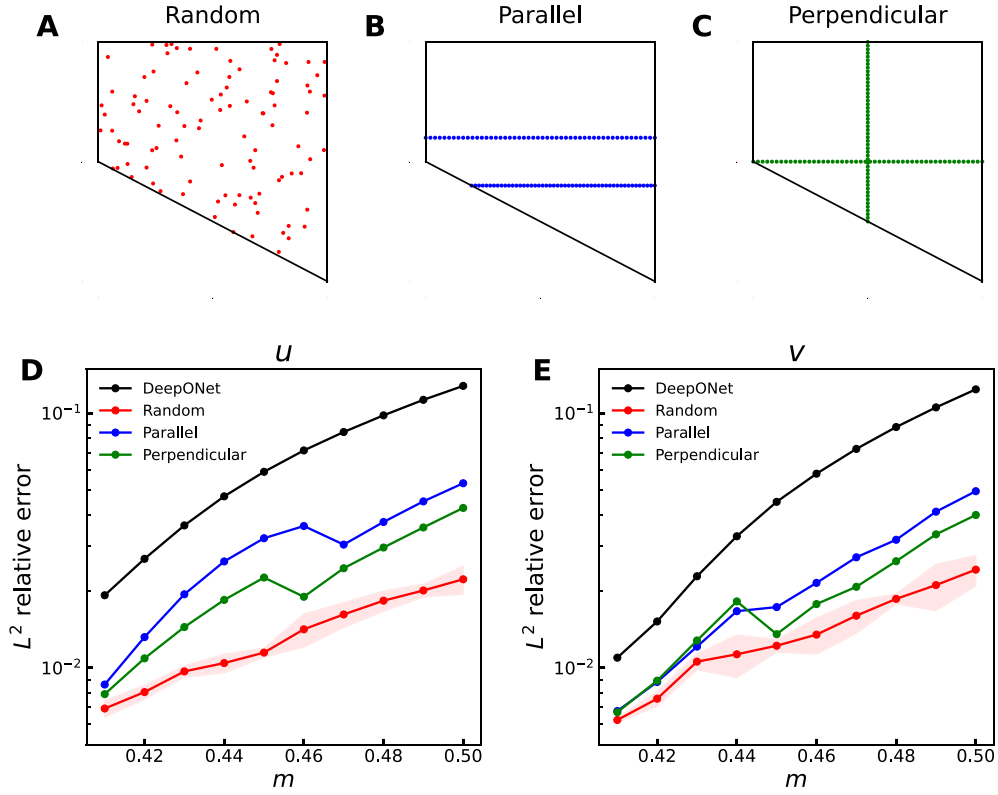
where  $\gamma_\lambda = 0.3$  is the learning rate. In this way, we increase the weight of the new information gradually during training. Introducing the adaptive adjustment of  $\lambda$  further slightly improves the accuracy (Figs. 7C and D).

We show an example for illustrating the prediction and absolute errors of all methods when  $m = 0.5$  in Figs. 7E and F. In this example, most proposed models exhibit better predictions of the velocity field than pre-trained DeepONet and GPR. Among these methods, fine-tuning with observations, either alone (FT-Obs-A) or together (FT-Obs-T), have particularly accurate prediction of the velocity field in both  $x$ - and  $y$ -component. However, multifidelity methods have larger error at the region near the central vortex and the moving lid in the  $x$ -direction. This results from the limitation that finite number of data points may perform weakly or even fail in fitting these locations with relatively large gradient.

We have tested DeepONet with different levels of extrapolation. Here, we cannot use the 2-Wasserstein distance to quantify the extrapolation complexity, as each test case is a single function rather than a function space. Hence,



**Fig. 7.** Lid-driven cavity flows in Section 4.5. (A and B) Examples of the lid-driven cavity flow in two geometries. (A)  $m = 0$ . (B)  $m = 0.5$ .  $Re = 1000$ . (C and D)  $L^2$  relative error of different methods for (C) the  $x$ -component of velocity  $u$  and (D) the  $y$ -component of velocity  $v$ . The left part ( $m \leq 0.40$ ) is interpolation, while the right part ( $m > 0.40$ ) is extrapolation. (E and F) An example of extrapolation ( $m = 0.5$ ). (E) Predictions (first row) and errors (second row) for  $u$ . (F) Predictions (first row) and errors (second row) for  $v$ . FT-Obs-T with adaptive weight (adaptive  $\lambda$ ) obtains the best result. (G)  $L^2$  relative error of DeepONet extrapolation with respect to the  $L^2$  distance between the training and testing functions.



**Fig. 8.** FT-Obs-T for different data observations cases. (A) Random sampling in the domain. (B) Sampling along two parallel lines. (C) Sampling along two perpendicular lines. (D) Extrapolation error of the  $x$ -component of velocity. (E) Extrapolation error of the  $y$ -component of velocity.

we directly measure the extrapolation complexity by the smallest  $L^2$  distance between the test function  $l(x)_{\text{test}}$  and the training functions  $l(x)_{\text{train}}$ , defined by

$$\min_{l(x)_{\text{train}}} \|l(x)_{\text{train}} - l(x)_{\text{test}}\|_2.$$

The extrapolation error grows with respect to the  $L^2$  distance. Specifically, for the  $x$ -component of velocity (Fig. 7G),

$$\text{Error} \propto (L^2)^{0.49},$$

and for the  $y$ -component of velocity (Fig. 7H), we have

$$\text{Error} \propto (L^2)^{0.66}.$$

For the experiments above, the new observations are randomly sampled in the domain (Fig. 8A), which might not be possible in practice. Hence, we also consider two more realistic cases, where 50 data observations are uniformly sampled in certain lines (Figs. 8B and C). For the “Parallel” case (Fig. 8B), the new data points come from two horizontal lines,  $y = 0.4$  and  $y = 0.6$ . For the “Perpendicular” case (Fig. 8C), the new data points come from a horizontal line and a vertical line,  $y = 0.5$  and  $x = 0.5$ . We use FT-Obs-T method for the three cases, each with 100 new observations. Random sampling in the domain leads to the most accurate prediction, while the other two sampling methods also achieve errors smaller than  $< 5\%$  (Figs. 8D and E).

**Table 3**  
**Comparison of extrapolation accuracy and inference speed for different methods.**  
 More stars (★) represent better accuracy and faster speed.

| New information     | Methods         | Extrapolation accuracy | Inference speed |
|---------------------|-----------------|------------------------|-----------------|
| Physics             | FT-Phys (Trunk) | ***                    | **              |
|                     | FT-Obs-A        | **                     | ***             |
| Sparse observations | FT-Obs-T        | ***                    | **              |
|                     | MF-GPR          | *                      | **              |
|                     | MF-NN           | **                     | *               |

## 5. Summary

Having a new input in the  $\text{Ex.}^+$  region is inevitable in real-world applications and would lead to large errors and failure of NNs. In this study, we first present a systematic study of extrapolation error of deep operator networks (DeepONets). We provide a quantitative definition of the extrapolation complexity by the 2-Wasserstein distance between two function spaces. Similar to interpolation error, we found a U-shaped error curve for extrapolation with respect to model capacity, such as network sizes and training iterations, but compared with interpolation and  $\text{Ex.}^-$ , the  $\text{Ex.}^+$  curve has larger error and earlier transition point. We also found that a larger training dataset is helpful for both interpolation and extrapolation.

To improve the prediction accuracy under  $\text{Ex.}^+$  scenarios, we consider additional information coming from physics or sparse observations. As the first step of the prediction workflow, we determine if the new input is in the region of interpolation or extrapolation. Given the governing partial differential equations (PDEs) of the system, we employ the PDE loss to fine-tune a pre-trained DeepONet (either the entire DeepONet or a part of the network). When we have extra sparse observations, we propose to either fine-tune a pre-trained DeepONet or apply a multifidelity learning framework. We demonstrate the excellent extrapolation capability of the proposed methods for diverse PDE problems. Furthermore, we validate the robustness of proposed methods by testing with different levels of noise.

We provide a practical guideline in choosing a proper extrapolation method depending on the available information and desired accuracy and inference speed in Table 3. When we have the physics as new information, fine-tuning with physics (FT-Phys) can achieve very high accuracy, and the computational cost of fine-tuning depends on the complexity of the PDEs. We found that only fine-tuning the trunk net usually works the best. We hypothesize that this is due to that the trunk net corresponds to the basis functions, which offers more flexibility than tuning the branch net (i.e., coefficients). For a complete understanding, more numerical and theoretical works for diverse PDEs should be done in the future. When we have sparse observations, fine-tuning with observations (FT-Obs-A and FT-Obs-T) usually has better accuracy than multifidelity learning methods (MF-GPR and MF-NN). As FT-Obs-A only takes a small number of new observations, FT-Obs-A has a faster inference speed than FT-Obs-T, but FT-Obs-A usually has lower accuracy than FT-Obs-T due to overfitting and catastrophic forgetting. As MF-GPR and MF-NN need to train a new model from scratch, they have a relatively slow inference speed.

This study is the first attempt to understand and address the important open issue of extrapolation of deep neural operators, and more work should be done both theoretically and computationally. We observed the U-shaped error curve for extrapolation, but there could exist a double-descent behavior when the model capacity is further increased, which will be investigated in future work. In this study, we consider either the complete physics or sparse observations as the new information, but in practice, we may have sparse observations with partial physics simultaneously, and a corresponding efficient method should be developed. On the theoretical side, there have been a few efforts to address interpolation errors for specific forms of the operator [9,64–69]. A theoretical understanding of extrapolation error could be even more challenging, and very limited work has been done so far [70].

**Table A.4**

Abbreviations and notations.

| In.                                     | Interpolation                                                             |
|-----------------------------------------|---------------------------------------------------------------------------|
| Ex. <sup>-</sup>                        | Extrapolation when $l_{\text{train}} < l_{\text{test}}$                   |
| Ex. <sup>+</sup>                        | Extrapolation when $l_{\text{train}} > l_{\text{test}}$                   |
| FT-Phys                                 | Fine-tune with physics in Section 3.3                                     |
| FT-Obs-A                                | Fine-tune with new observations alone in Section 3.4                      |
| FT-Obs-T                                | Fine-tune with training data and new observations together in Section 3.4 |
| MF-NN                                   | Multifidelity neural networks in Section 3.5                              |
| MF-GPR                                  | Multifidelity Gaussian process regression in Section 3.5                  |
| $l$                                     | Correlation length of Gaussian random field                               |
| $\mathcal{G}$                           | operator to learn                                                         |
| $\tilde{\mathcal{G}}$                   | Pre-trained DeepONet                                                      |
| $\Omega$                                | Domain of the PDE                                                         |
| $\{x_1, x_2, \dots, x_m\}$              | Scattered sensors                                                         |
| $[v(x_1), v(x_2), \dots, v(x_m)]$       | Input of branch network                                                   |
| $[b_1(v), b_2(v), \dots, b_p(v)]$       | Output of branch network                                                  |
| $\xi$                                   | Input of trunk network                                                    |
| $[t_1(\xi), t_2(\xi), \dots, t_p(\xi)]$ | Outputs of trunk network, where $p$ is the number of neurons              |
| $\tilde{u}$                             | Prediction using pre-trained DeepONet                                     |
| $W_2$                                   | 2-Wasserstein distance                                                    |
| $\mathcal{F}$                           | Governing PDEs and/or physical constraints                                |
| $\mathcal{B}$                           | Initial and boundary conditions                                           |
| $\mathcal{D}$                           | Sparse observations                                                       |
| $\mathcal{E}_{\text{phys}}$             | Mismatch error of physics                                                 |
| $\mathcal{E}_{\text{obs}}$              | Mismatch error of observations                                            |
| $\mathcal{L}_{\text{phys}}$             | Loss for FT-Phys                                                          |
| $\mathcal{L}_{\mathcal{F}}$             | Loss of PDE residuals                                                     |
| $\mathcal{L}_{\mathcal{B}}$             | Loss of initial and boundary conditions                                   |
| $\mathcal{L}_{\text{obs}}$              | Loss for FT-Obs-A                                                         |
| $\mathcal{L}_{\mathcal{F}, \text{obs}}$ | Loss for FT-Obs-T                                                         |
| $w_{\mathcal{F}}, w_{\mathcal{B}}$      | Weights in FT-Phys loss                                                   |
| $\lambda$                               | Weight in FT-Obs-T loss                                                   |

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

No data was used for the research described in the article.

## Acknowledgments

This work was supported by the U.S. Department of Energy [DE-SC0022953] and OSD/AFOSR MURI, USA grant FA9550-20-1-0358.

## Appendix A. Abbreviations and notations

We list in Table A.4 the main abbreviations and notations that are used throughout this paper.

## Appendix B. Hyperparameters

Table B.5 provides the DeepONet architectures used in all examples and the hyperparameters for training.

For the method of fine-tuning with physics, we use the Adam optimizer and the number of iterations is listed in Table B.6. For FT-Obs-A, we fine-tune the DeepONet for 500 iterations using the L-BFGS optimizer for all the

**Table B.5**

**DeepONet architectures and the hyperparameters used for pre-training.** In the “Depth” and “Activation” columns, the first and second subcolumns correspond to the trunk and branch net, respectively. The branch net and trunk net use the same network width.

| Problems                                  | Depth | Width | Activation | Learning rate | Iterations      |
|-------------------------------------------|-------|-------|------------|---------------|-----------------|
| <a href="#">Appendix C</a> Antiderivative | 3, 3  | 40    | tanh, ReLU | 0.005         | $5 \times 10^4$ |
| Section 4.1 Diffusion-reaction            | 4, 3  | 100   | GELU, ReLU | 0.001         | $5 \times 10^5$ |
| Section 4.2 Burgers’                      | 4, 3  | 100   | GELU, ReLU | 0.001         | $5 \times 10^5$ |
| Section 4.3 Advection                     | 4, 3  | 100   | GELU, ReLU | 0.001         | $5 \times 10^5$ |
| Section 4.4 Poisson (Notch)               | 4, 3  | 100   | GELU, ReLU | 0.001         | $5 \times 10^5$ |
| Section 4.5 Lid-driven cavity             | 4, 3  | 100   | GELU, ReLU | 0.001         | $5 \times 10^5$ |

**Table B.6**

Hyperparameters for FT-Phys.

|                                           | FT-Phys iterations |
|-------------------------------------------|--------------------|
| <a href="#">Appendix C</a> Antiderivative | 1000               |
| Section 4.1 Diffusion-reaction            | 2000               |
| Section 4.2 Burgers’                      | 5000               |
| Section 4.3 Advection                     | 5000               |

**Table B.7**

**Hyperparameters for MF-NN.** In the columns of low- and high-fidelity networks, the first and second numbers are depth and width, respectively.

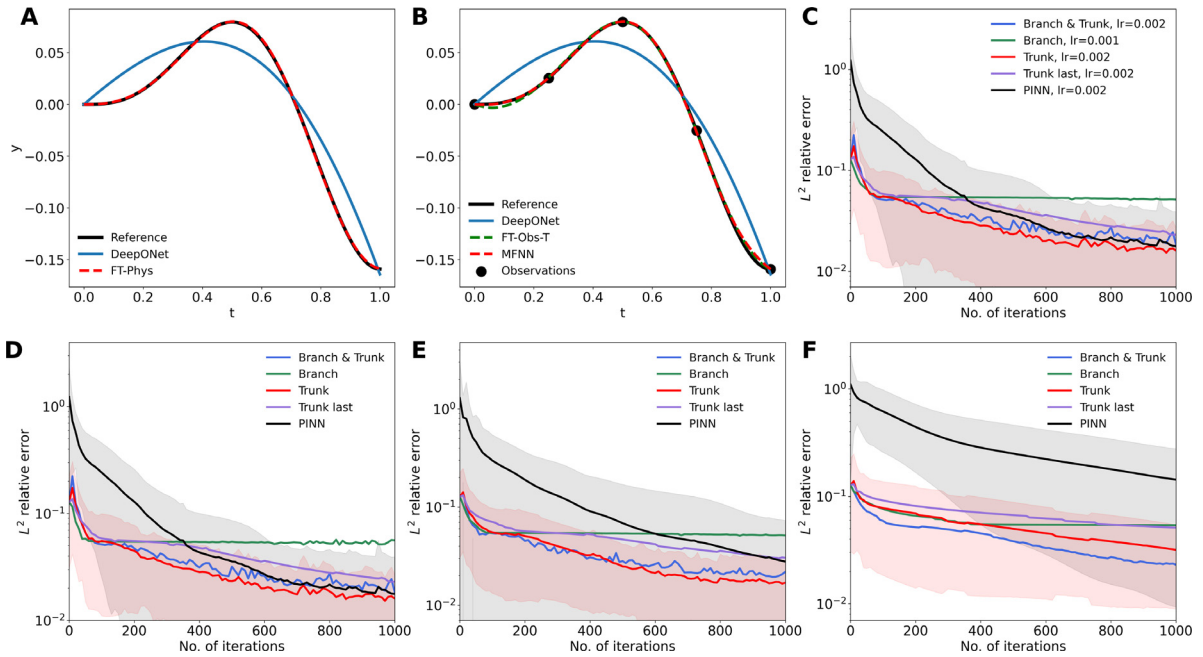
|                                           | $ S $ | Low-fidelity network | High-fidelity network | Learning rate |
|-------------------------------------------|-------|----------------------|-----------------------|---------------|
| <a href="#">Appendix C</a> Antiderivative | 100   | 4, 40                | 3, 30                 | 0.005         |
| Section 4.1 Diffusion-reaction            | 10201 | 4, 128               | 3, 15                 | 0.001         |
| Section 4.2 Burgers’                      | 10201 | 4, 128               | 3, 15                 | 0.001         |
| Section 4.3 Advection                     | 10201 | 4, 128               | 3, 15                 | 0.001         |
| Section 4.4 Poisson (Notch)               | 5082  | 4, 128               | 3, 15                 | 0.001         |
| Section 4.5 Lid-driven cavity             | 10201 | 4, 128               | 3, 15                 | 0.001         |

problems, except that for the antiderivative problem, we use the Adam optimizer with a learning rate of 0.001 for 1000 iterations. For FT-Obs-T, we choose  $\lambda = 0.3$  for all cases, and we fine-tune the DeepONet for 3000 iterations using the Adam optimizer with the learning rate of 0.001, except that for the antiderivative problem, we train for 1000 iterations.

For multifidelity learning, the size of the low-fidelity dataset  $S$  is in [Table B.7](#). However, MF-GPR is not able to handle a large dataset, and thus we use at most 400 low-fidelity data points. For MF-NN, we use the SiLU activation function, and the network size is in [Table B.7](#). We train MF-NN using the Adam optimizer for 10000 iterations. Also, a  $L^2$  regularization is applied, and the strength is  $10^{-6}$  for the antiderivative operator. For other problems, the strength is  $10^{-5}$ ,  $10^{-6}$ ,  $10^{-7}$ , and  $10^{-8}$  for 20, 50, 100, and 200 high-fidelity data points, respectively.

## Appendix C. Antiderivative operator

We also consider the antiderivative operator in [Section 2.2](#). The goal is to learn the operator mapping from  $v(x)$  to the solution  $u(x)$ . For the training dataset,  $v(x)$  is sampled from GRF of an RBF kernel with the correlation length  $l_{\text{train}} = 0.5$ . To test the Ex.<sup>+</sup>, we generate a test dataset of 100 functions with  $l_{\text{test}} = 0.2$ .



**Fig. C.9.** Antiderivative operator in [Appendix C](#). (A) Predictions of the pre-trained DeepONet and fine-tuning with physics. (B) Predictions of the pre-trained DeepONet, FT-Obs-T, and MF-NN with 5 new observations. (C) The best result of each method among different learning rates. (D, E, and F) Training trajectories under the learning rate of (D) 0.002, (E) 0.001, and (F) 0.0002. The curves and shaded regions represent the mean and one standard deviation of 100 test cases. FT-Phys and MF-NN obtain the best results. (For clarity, only some standard deviations are plotted.)

**Table D.8**

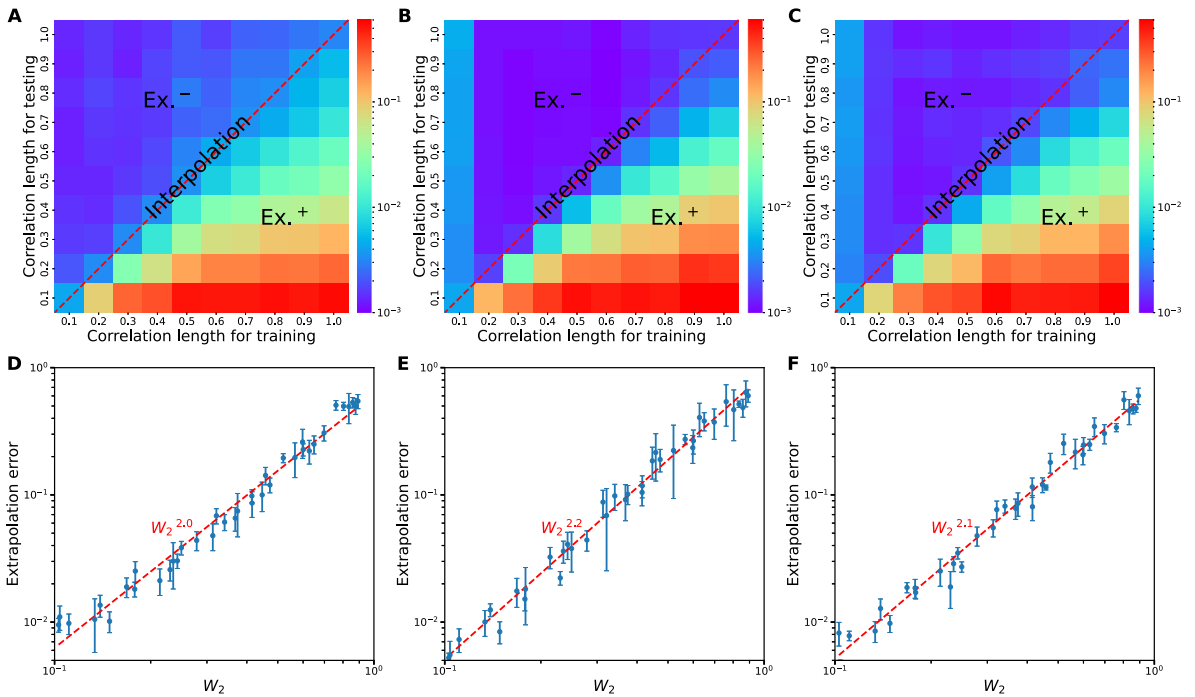
$L^2$  relative error of different methods for the antiderivative operator in [Appendix C](#).  $l_{\text{train}} = 0.5$  and  $l_{\text{test}} = 0.2$ . Bold font indicates the smallest two errors in each case, and the underlined text indicates the smallest error.

|             |                                            |                                            |                                            |                                            |
|-------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|
| DeepONet    | Error(In.): $0.93 \pm 0.20\%$              |                                            | Error(Ex.*): $11.6 \pm 5.26\%$             |                                            |
| PI-DeepONet | Error(In.): $0.81 \pm 0.69\%$              |                                            | Error(Ex.*): $17.4 \pm 6.01\%$             |                                            |
| <hr/>       |                                            |                                            |                                            |                                            |
| PINN        | $1.77 \pm 2.14\%$                          |                                            |                                            |                                            |
|             | Branch & Trunk                             | Branch                                     | Trunk                                      | Trunk last one                             |
| FT-Phys     | <b><u><math>1.84 \pm 1.49\%</math></u></b> | $5.09 \pm 3.66\%$                          | <b><u><math>1.52 \pm 1.00\%</math></u></b> | $2.29 \pm 1.65\%$                          |
| <hr/>       |                                            |                                            |                                            |                                            |
|             | 4 points                                   | 5 points                                   | 6 points                                   | 7 points                                   |
| GPR         | $14.7 \pm 16.7\%$                          | $8.63 \pm 14.1\%$                          | $4.98 \pm 12.8\%$                          | $3.15 \pm 12.4\%$                          |
| FT-Obs-A    | $10.7 \pm 7.70\%$                          | $5.98 \pm 4.15\%$                          | $4.45 \pm 3.30\%$                          | $4.22 \pm 3.11\%$                          |
| FT-Obs-T    | <b><u><math>9.05 \pm 6.50\%</math></u></b> | $5.61 \pm 4.19\%$                          | $2.92 \pm 2.17\%$                          | $1.87 \pm 1.24\%$                          |
| MF-GPR      | $13.0 \pm 11.2\%$                          | <b><u><math>5.17 \pm 3.74\%</math></u></b> | <b><u><math>2.49 \pm 1.71\%</math></u></b> | <b><u><math>1.15 \pm 0.99\%</math></u></b> |
| MF-NN       | <b><u><math>8.99 \pm 5.83\%</math></u></b> | <b><u><math>4.79 \pm 3.02\%</math></u></b> | <b><u><math>2.10 \pm 1.33\%</math></u></b> | <b><u><math>1.35 \pm 0.99\%</math></u></b> |

**Table D.9**

$L^2$  relative error of different methods for the diffusion-reaction equation in Section 4.1.  $l_{\text{train}} = 0.5$  and  $l_{\text{test}} = 0.2$ . Bold font indicates the smallest two errors in each case, and the underlined text indicates the smallest error.

|             |                                            |                                            |                                            |                                            |
|-------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|
| DeepONet    | Error(In.): $0.74 \pm 0.29\%$              |                                            | Error(Ex.+): $10.4 \pm 6.24\%$             |                                            |
| PI-DeepONet | Error(In.): $0.42 \pm 0.20\%$              |                                            | Error(Ex.+): $10.2 \pm 6.30\%$             |                                            |
| PINN        |                                            |                                            | $0.49 \pm 0.24\%$                          |                                            |
| FT-Phys     | Branch & Trunk                             | Branch                                     | Trunk                                      | Trunk last one                             |
|             | $0.37 \pm 0.16\%$                          | $1.28 \pm 0.62\%$                          | <b><math>0.32 \pm 0.15\%</math></b>        | $0.48 \pm 0.21\%$                          |
|             |                                            |                                            |                                            | <b><u><math>0.28 \pm 0.14\%</math></u></b> |
|             | 20 points                                  | 50 points                                  | 100 points                                 | 200 points                                 |
| GPR         | $34.5 \pm 15.0\%$                          | $9.63 \pm 4.55\%$                          | $2.59 \pm 1.57\%$                          | <b><u><math>0.61 \pm 0.39\%</math></u></b> |
| FT-Obs-A    | <b><math>5.51 \pm 3.08\%</math></b>        | <b><math>3.36 \pm 1.72\%</math></b>        | $2.41 \pm 1.19\%$                          | $1.63 \pm 0.69\%$                          |
| FT-Obs-T    | <b><u><math>4.56 \pm 2.66\%</math></u></b> | <b><u><math>2.69 \pm 1.51\%</math></u></b> | <b><u><math>1.83 \pm 0.86\%</math></u></b> | $1.32 \pm 0.59\%$                          |
| MF-GPR      | $12.2 \pm 5.07\%$                          | $7.96 \pm 3.45\%$                          | <b><u><math>2.26 \pm 1.49\%</math></u></b> | <b><u><math>0.48 \pm 0.27\%</math></u></b> |
| MF-NN       | $7.86 \pm 5.18\%$                          | $4.50 \pm 2.20\%$                          | $2.73 \pm 1.18\%$                          | $1.50 \pm 0.68\%$                          |



**Fig. D.10.**  $L^2$  relative errors of DeepONet trained and tested on datasets with different correlation lengths for the diffusion-reaction equation. (A and D)  $k = 0.01$  and  $D = 1.0$ . (B and E) Reaction term  $ku$  is linear with  $k = 0.01$  and  $D = 0.01$ . (C and F) Reaction term  $ku^3$  is cubic with  $k = 0.01$  and  $D = 0.01$ . (A, B, and C) Testing error for different pairs of training and testing functions. (D, E, and F) The Ex.+ error grows with a polynomial rate with respect to the  $W_2$  distance between the training and test spaces. The error is the mean of 10 runs, and the error bars represent one standard deviation.

**Table D.10**

$L^2$  relative error of different methods for the Burgers' equation in Section 4.2.  $l_{\text{train}} = 1.0$  and  $l_{\text{test}} = 0.6$ . Bold font indicates the smallest two errors in each case, and the underlined text indicates the smallest error.

|             |                                     |                                     |                                             |                                     |                   |
|-------------|-------------------------------------|-------------------------------------|---------------------------------------------|-------------------------------------|-------------------|
| DeepONet    | Error(In.): $2.21 \pm 1.11\%$       |                                     | Error(Ex. <sup>+</sup> ): $6.53 \pm 3.33\%$ |                                     |                   |
| PI-DeepONet | Error(In.): $4.96 \pm 1.80\%$       |                                     | Error(Ex. <sup>+</sup> ): $9.27 \pm 3.93\%$ |                                     |                   |
| PINN        |                                     |                                     | $1.34 \pm 0.59\%$                           |                                     |                   |
|             | Branch & Trunk                      | Branch                              | Trunk                                       | Trunk last one                      | Trunk last two    |
| FT-Phys     | <b><math>0.74 \pm 0.36\%</math></b> | $6.82 \pm 3.24\%$                   | <b><math>0.68 \pm 0.39\%</math></b>         | $0.94 \pm 0.54\%$                   | $0.76 \pm 0.37\%$ |
|             | 20 points                           | 50 points                           | 100 points                                  | 200 points                          |                   |
| GPR         | $43.0 \pm 20.9\%$                   | $29.2 \pm 15.1\%$                   | $18.6 \pm 10.3\%$                           | $12.8 \pm 7.58\%$                   |                   |
| FT-Obs-A    | <b><math>4.97 \pm 2.57\%</math></b> | <b><math>4.40 \pm 2.27\%</math></b> | <b><math>4.00 \pm 1.99\%</math></b>         | <b><math>3.78 \pm 2.02\%</math></b> |                   |
| FT-Obs-T    | <b><math>4.53 \pm 2.32\%</math></b> | <b><math>3.89 \pm 1.98\%</math></b> | <b><math>3.44 \pm 1.72\%</math></b>         | <b><math>3.07 \pm 1.55\%</math></b> |                   |
| MF-GPR      | $6.28 \pm 3.20\%$                   | $5.79 \pm 3.29\%$                   | $5.15 \pm 2.97\%$                           | $4.22 \pm 2.31\%$                   |                   |
| MF-NN       | $6.34 \pm 3.23\%$                   | $5.62 \pm 2.84\%$                   | $5.15 \pm 2.56\%$                           | $4.59 \pm 2.33\%$                   |                   |

**Table D.11**

$L^2$  relative error of different methods for the advection equation in Section 4.3.  $l_{\text{train}} = 0.5$  and  $l_{\text{test}} = 0.2$ . Bold font indicates the smallest two errors in each case, and the underlined text indicates the smallest error.

|             |                                     |                                     |                                            |                                     |                                     |
|-------------|-------------------------------------|-------------------------------------|--------------------------------------------|-------------------------------------|-------------------------------------|
| DeepONet    | Error(In.): $1.30 \pm 0.26\%$       |                                     | Error(Ex.*): $8.75 \pm 6.42\%$             |                                     |                                     |
| PI-DeepONet | Error(In.): $3.60 \pm 0.70\%$       |                                     | Error(Ex.*): $10.4 \pm 4.48\%$             |                                     |                                     |
| PINN        |                                     |                                     | $1.67 \pm 0.53\%$                          |                                     |                                     |
|             | Branch & Trunk                      | Branch                              | Trunk                                      | Trunk last one                      | Trunk last two                      |
| FT-Phys     | $1.05 \pm 0.30\%$                   | $2.65 \pm 1.17\%$                   | <u><b><math>0.93 \pm 0.23\%</math></b></u> | <b><math>1.01 \pm 0.31\%</math></b> | <b><math>0.99 \pm 0.26\%</math></b> |
|             | 20 points                           | 50 points                           | 100 points                                 | 200 points                          |                                     |
| GPR         | $34.6 \pm 9.46\%$                   | $25.8 \pm 7.89\%$                   | $16.7 \pm 3.99\%$                          | $11.6 \pm 3.54\%$                   |                                     |
| FT-Obs-A    | <b><math>6.15 \pm 2.91\%</math></b> | <b><math>5.79 \pm 2.43\%</math></b> | <b><math>3.91 \pm 1.52\%</math></b>        | <b><math>2.42 \pm 0.79\%</math></b> |                                     |
| FT-Obs-T    | <b><math>5.07 \pm 2.34\%</math></b> | <b><math>4.08 \pm 1.73\%</math></b> | <b><math>3.16 \pm 1.34\%</math></b>        | <b><math>2.00 \pm 0.74\%</math></b> |                                     |
| MF-GPR      | $8.39 \pm 5.38\%$                   | $6.80 \pm 4.00\%$                   | $5.47 \pm 2.87\%$                          | $4.19 \pm 2.01\%$                   |                                     |
| MF-NN       | $8.46 \pm 4.72\%$                   | $6.20 \pm 2.81\%$                   | $6.90 \pm 6.47\%$                          | $4.64 \pm 3.55\%$                   |                                     |

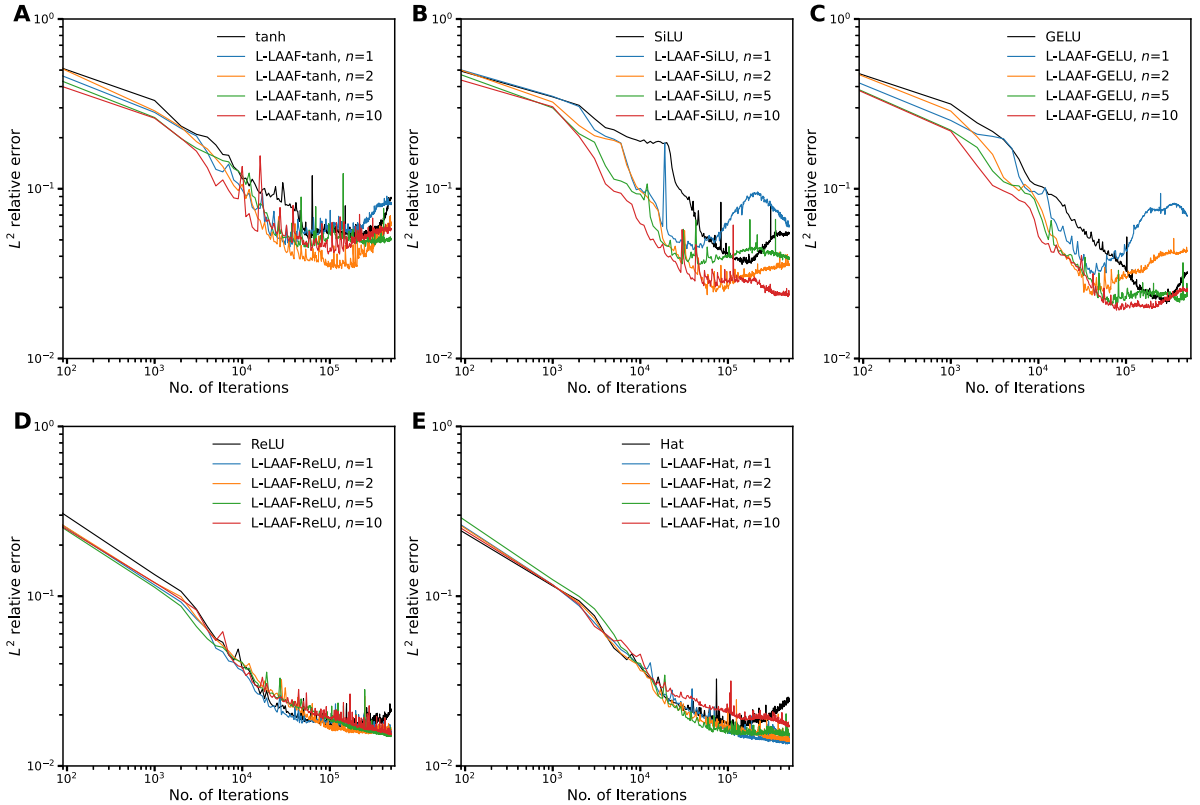
Table D.8 summarizes the results of different methods. The pre-trained DeepONet has an average  $L^2$  relative error of 11.6%, and PI-DeepONet has an average  $L_2$  relative error of 7.44% for the test dataset. When we have information of physics, fine-tuning with physics achieves accuracy of 1.52%, which is more accurate than PINN. When we have more than 5 sparse observations, FT-Obs-T, MF-GPR and MF-NN achieve accuracy about 2%, but FT-Obs-A has a relatively large error. In this case, multifidelity learning (MF-GPR and MF-NN) outperforms FT-Obs-A and FT-Obs-T. We note that this example is a relatively simple problem and multifidelity methods work well, but for the other examples, fine-tuning with new observations has better accuracy. Moreover, an example of prediction result is also provided in Figs. C.9A and B to illustrate the enhancement resulted from employing proposed methods. While the performance of FT-Obs-A, FT-Obs-T, MF-NN, and MF-GP are similar, only FT-Obs-T and MF-NN are plotted. Irrespective of new physical information (Fig. C.9A) or new observations (Fig. C.9B), the prediction results confirm that the proposed methods ameliorate the effect of Ex.\* to a great extent.

*Detailed results of FT-Phys..* For fine-tuning with physics, we test three different learning rates (i.e., 0.002, 0.001, 0.0002). For both PINN and fine-tuning with physics, we enforce the hard constraint for the initial condition. Among all variants of FT-Phys, fine-tuning the trunk net with a learning rate of 0.002 performs the best (Fig. C.9C). The performance of fine-tuning with physics for different learning rates is exhibited in Figs. C.9D, E, and F. Fine-tuning the trunk net and the entire DeepONet perform similarly well when the learning rate is large, and the full net slightly outperforms the trunk net as the learning rate is as small as 0.0002.

**Table D.12**

$L^2$  relative error of different methods for the Poisson equation in Section 4.4.  $l_{\text{train}} = 0.5$  and  $l_{\text{test}} = 0.2$ . Bold font indicates the smallest two errors in each case, and the underlined text indicates the smallest error.

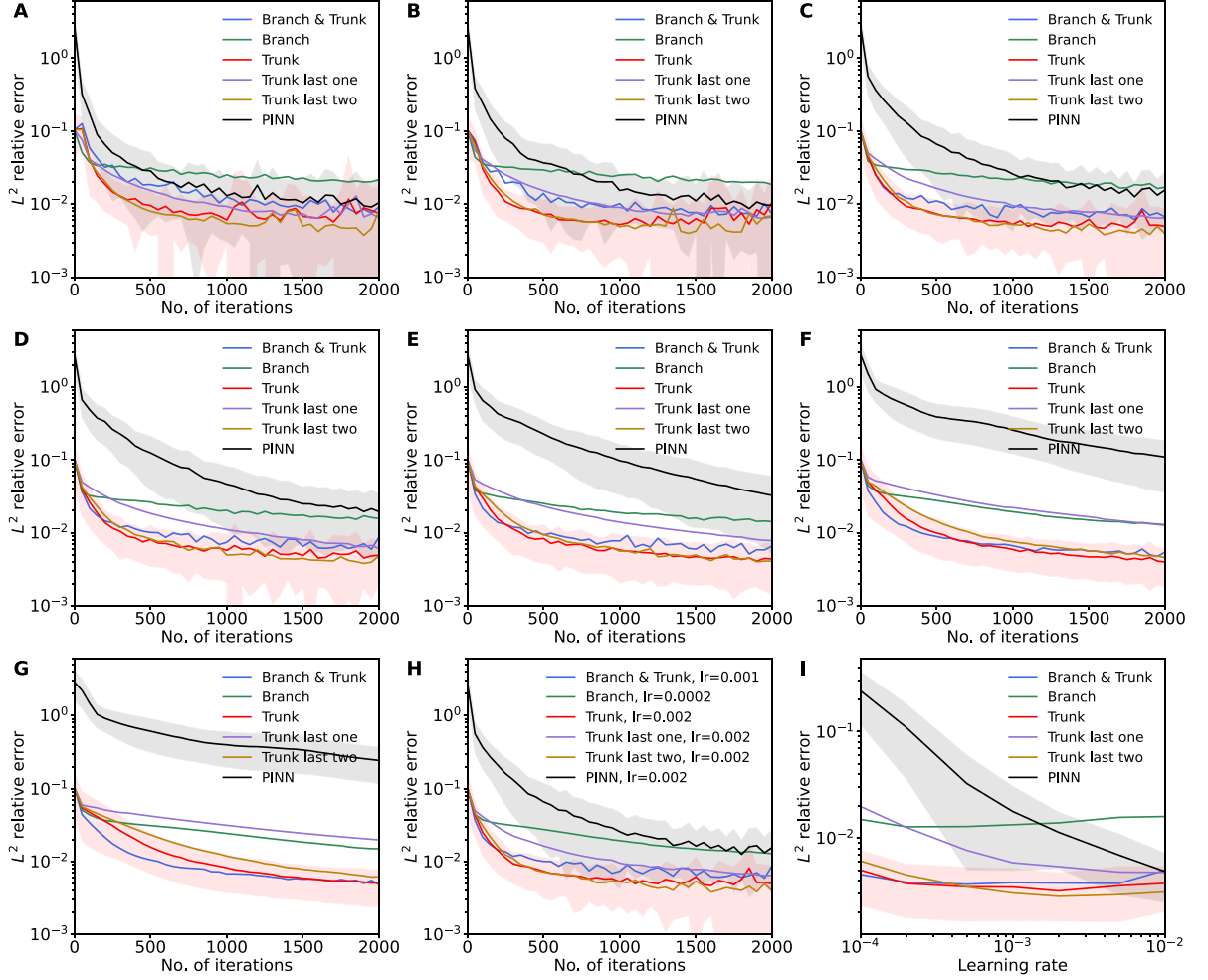
| DeepONet | Error(In.): $0.09 \pm 0.04\%$       |                                     | Error(Ex. <sup>+</sup> ): $14.8 \pm 12.0\%$ |                                     |
|----------|-------------------------------------|-------------------------------------|---------------------------------------------|-------------------------------------|
|          | 20 points                           | 50 points                           | 100 points                                  | 200 points                          |
| GPR      | $16.7 \pm 9.51\%$                   | $12.1 \pm 6.57\%$                   | $8.06 \pm 4.99\%$                           | $6.26 \pm 4.63\%$                   |
| FT-Obs-A | <b><math>7.52 \pm 5.57\%</math></b> | <b><math>4.15 \pm 2.89\%</math></b> | <b><math>2.49 \pm 1.64\%</math></b>         | <b><math>1.80 \pm 1.06\%</math></b> |
| FT-Obs-T | <b><math>4.97 \pm 3.14\%</math></b> | <b><math>3.01 \pm 2.11\%</math></b> | <b><math>1.55 \pm 1.08\%</math></b>         | <b><math>0.95 \pm 0.56\%</math></b> |
| MF-GPR   | $8.00 \pm 5.58\%$                   | $5.25 \pm 3.38\%$                   | $3.38 \pm 1.97\%$                           | $2.42 \pm 1.39\%$                   |
| MF-NN    | $11.3 \pm 6.96\%$                   | $7.63 \pm 5.16\%$                   | $4.64 \pm 2.75\%$                           | $3.12 \pm 1.51\%$                   |

**Fig. E.11.** Section 2.5.3: L-LAAF with different scaling factors  $n$ . (A) tanh. (B) SiLU. (C) GELU. (D) ReLU. (E) Hat.

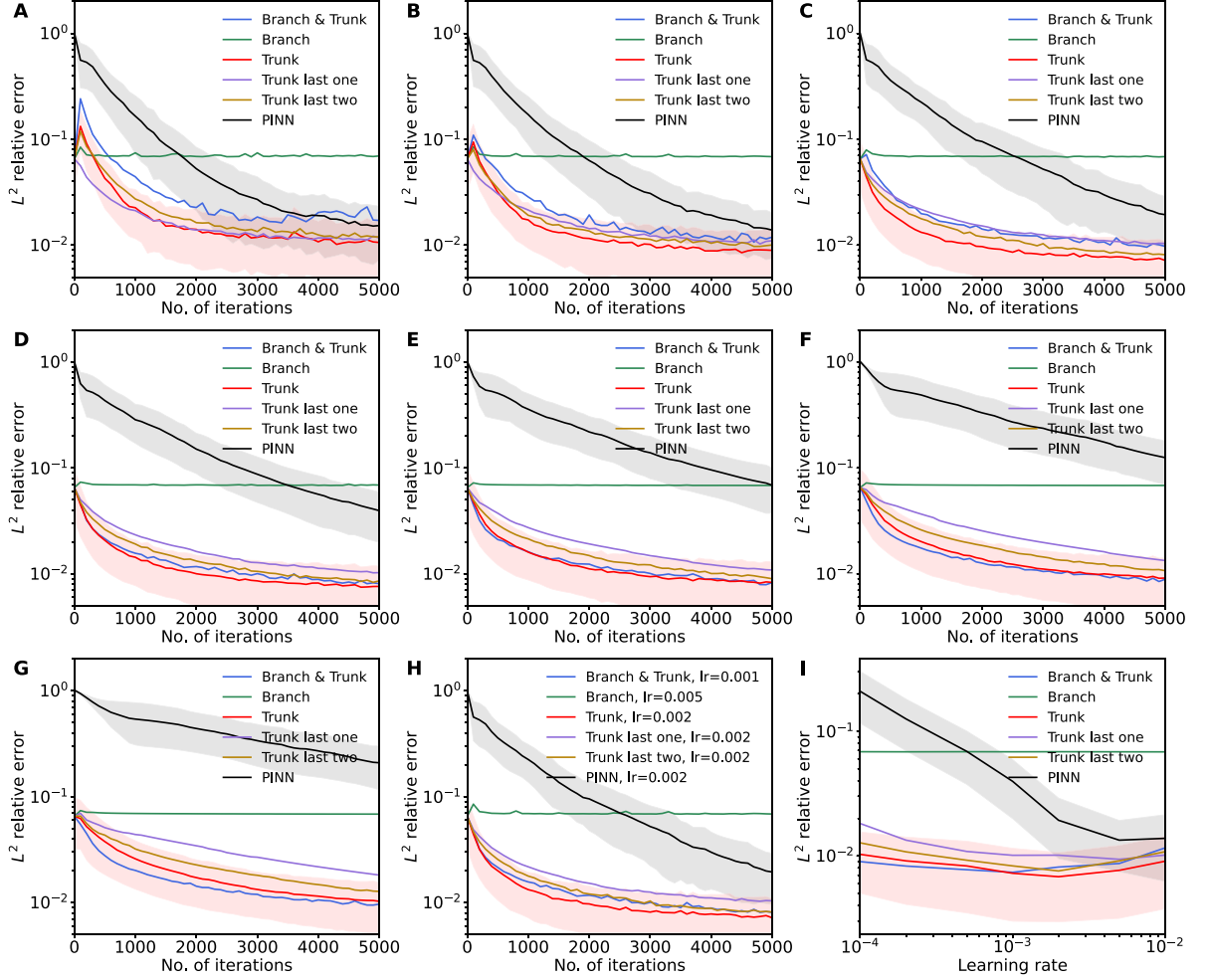
## Appendix D. More benchmark results

Fig. D.10 shows three more cases for the diffusion-reaction equation, which have a similar tendency as Fig. 1.

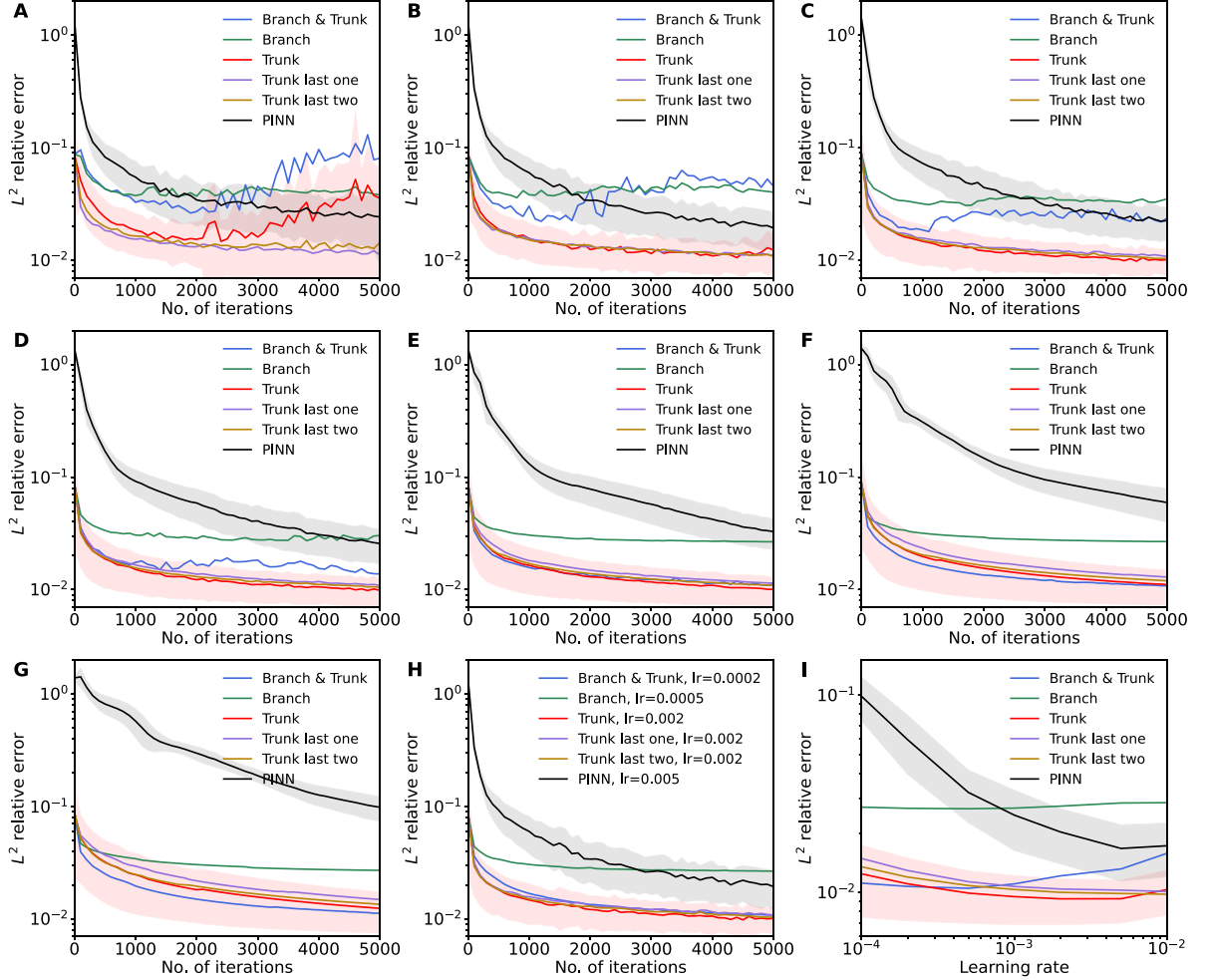
The following tables includes more results of the antiderivative operator (Table D.8), the diffusion-reaction equation (Table D.9), the Burgers' equation (Table D.10), the advection equation (Table D.11), and the Poisson equation (Table D.12).



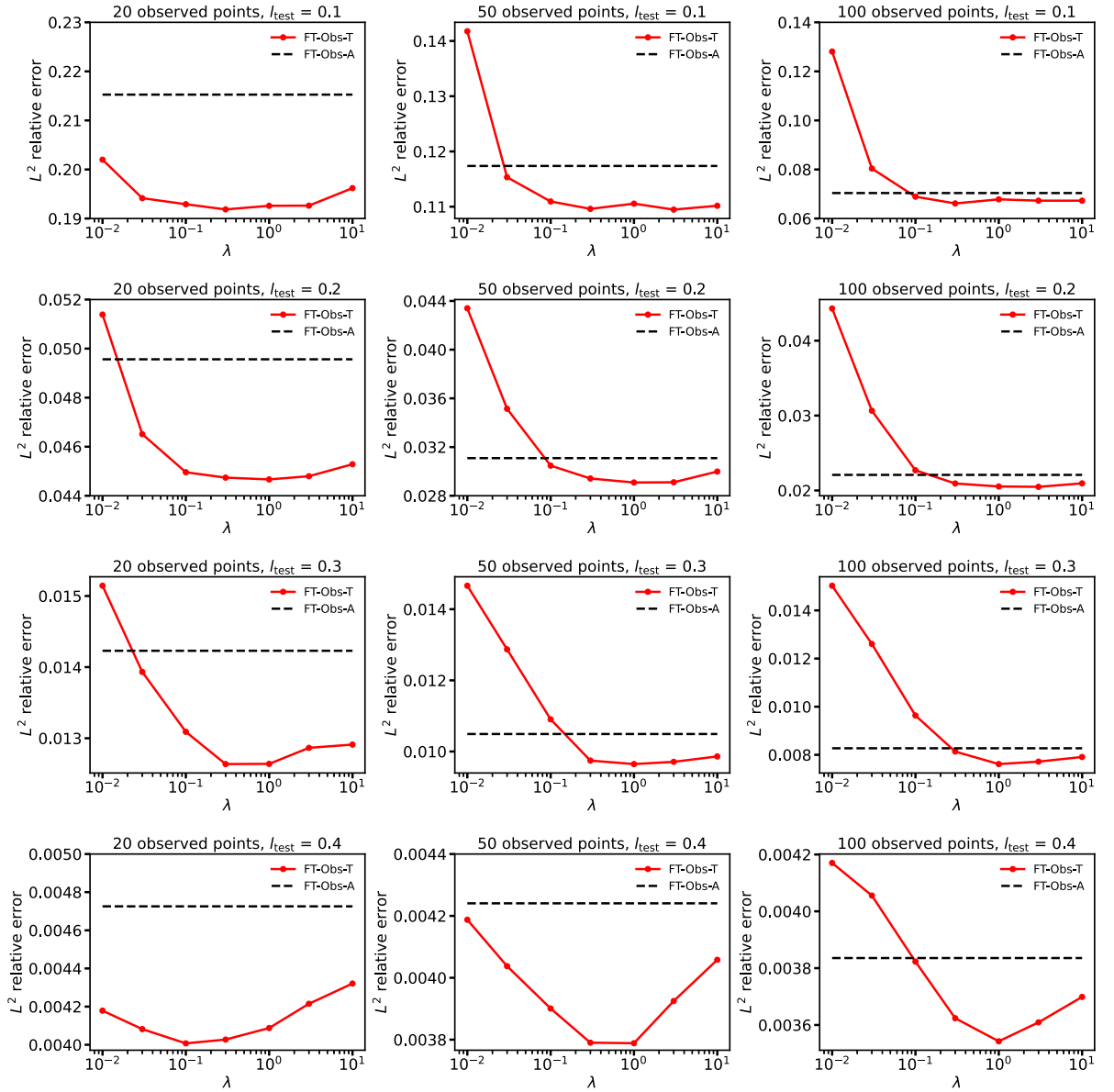
**Fig. F.12.** Section 4.1: Fine-tuning with physics for the diffusion-reaction equation. (A–G) Training trajectories under different learning rate of (A) 0.01, (B) 0.005, (C) 0.002, (D) 0.001, (E) 0.0005, (F) 0.0002, and (G) 0.0001. (H) The best result of each method among different learning rates. (I)  $L^2$  relative errors with respect to learning rate for fine-tuning different parts of DeepONet. The curves and shaded regions represent the mean and one standard deviation of 100 runs. For clarity, only standard deviations of trunk mode and PINN mode are plotted.



**Fig. F.13.** Section 4.2: Fine-tuning with physics for the Burgers' equation. (A–G) Training trajectories under different learning rate of (A) 0.01, (B) 0.005, (C) 0.002, (D) 0.001, (E) 0.0005, (F) 0.0002, and (G) 0.0001. (H) The best result of each method among different learning rates. (I)  $L^2$  relative errors with respect to learning rate for fine-tuning different parts of DeepONet. The curves and shaded regions represent the mean and one standard deviation of 100 runs. For clarity, only standard deviations of trunk mode and PINN mode are plotted.



**Fig. F.14.** Section 4.3: Fine-tuning with physics for the advection equation. (A–G) Training trajectories under different learning rate of (A) 0.01, (B) 0.005, (C) 0.002, (D) 0.001, (E) 0.0005, (F) 0.0002, and (G) 0.0001. (H) The best result of each method among different learning rates. (I)  $L^2$  relative errors with respect to learning rate for fine-tuning different parts of DeepONet. The curves and shaded regions represent the mean and one standard deviation of 100 runs. For clarity, only standard deviations of trunk mode and PINN mode are plotted.



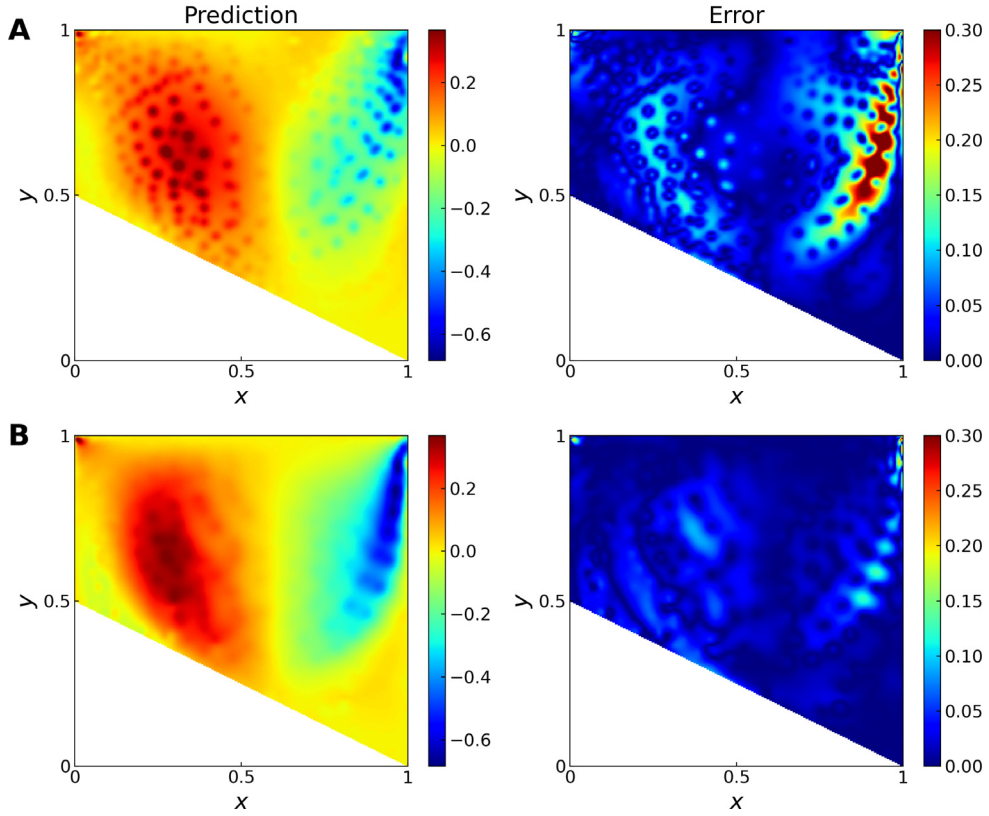
**Fig. G.15.** Section 4.1: Comparisons of different values of  $\lambda$  for FT-Obs-T method under different number of observed points and testing correlation lengths for the diffusion-reaction equation.

## Appendix E. Layer-wise locally adaptive activation function

For L-LAAF in Section 2.5.3, the scaling factor  $n$  is a hyperparameter to be tuned. We choose  $n$  from 1, 2, 5, and 10 for each activation function. The best scaling factors  $n$  for tanh, SiLU, GELU, ReLU, and Hat are 2, 10, 10, 5, and 1, respectively (Fig. E.11).

## Appendix F. Fine-tune with physics

With physics as additional information, fine-tuning with physics is considered and different learning rates (i.e., 0.01, 0.005, 0.002, 0.001, 0.0005, 0.0002, and 0.0001) are used to fine-tune the pre-trained DeepONet. Below are detailed results of diffusion-reaction equation (Fig. F.12), Burgers' equation (Fig. F.13), and advection equation (Fig. F.14).



**Fig. G.16.** Section 4.5: Prediction and error of MF-GPR with the RBF and Matern kernels for the lid-driven cavity flow. (A) The RBF kernel. (B) The Matern kernel with  $\nu = 1.5$ .

## Appendix G. Comparisons of different values of $\lambda$ for FT-Obs-T methods

For the diffusion-reaction equation in Section 4.1, we determine the effect of  $\lambda$  on test errors for different numbers of observed points and  $l_{\text{test}}$ . The results of 12 cases are shown in Fig. G.15.

## Appendix H. MF-GPR for the Lid-driven cavity flow

In most examples, the RBF kernel works well for MF-GPR. However, in the cavity flow problem, MF-GPR with the RBF kernel has a large error (Fig. G.16A), while the Matern kernel with  $\nu = 1.5$  performs better (Fig. G.16B). The Matern kernel is given by

$$k(x_1, x_2) = \frac{1}{\Gamma(\nu)2^{\nu-1}} \left( \frac{\sqrt{2\nu}}{l} \|x_1 - x_2\| \right)^\nu K_\nu \left( \frac{\sqrt{2\nu}}{l} \|x_1 - x_2\| \right),$$

where  $l$  is the correlation length,  $K_\nu(\cdot)$  is a modified Bessel function, and  $\Gamma(\cdot)$  is the gamma function.

## References

- [1] Kurt Hornik, Maxwell Stinchcombe, Halbert White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (5) (1989) 359–366.
- [2] Nathan Baker, Frank Alexander, Timo Bremer, Aric Hagberg, Yannis Kevrekidis, Habib Najm, Manish Parashar, Abani Patra, James Sethian, Stefan Wild, et al., Workshop report on basic research needs for scientific machine learning: Core technologies for artificial intelligence, in: Technical report, USDOE Office of Science (SC), Washington, DC (United States), 2019.
- [3] George Em Karniadakis, Ioannis G Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, Liu Yang, Physics-informed machine learning, *Nat. Rev. Phys.* 3 (6) (2021) 422–440.

- [4] Maziar Raissi, Paris Perdikaris, George Em Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707, <http://dx.doi.org/10.1016/j.jcp.2018.10.045>, <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [5] Lu Lu, Xuhui Meng, Zhiping Mao, George Em Karniadakis, DeepXDE: A deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228, <http://dx.doi.org/10.1137/19M1274067>.
- [6] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, Lu Lu, A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks, 2022, arXiv preprint [arXiv:2207.10289](https://arxiv.org/abs/2207.10289).
- [7] Tianping Chen, Hong Chen, Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems, *IEEE Trans. Neural Netw.* 6 (4) (1995) 911–917.
- [8] Lu Lu, Pengzhan Jin, Guofei Pang, Handy Zang, George Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (2021) 218–229, <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- [9] Beichuan Deng, Yeonjong Shin, Lu Lu, Zhongqiang Zhang, George Em Karniadakis, Approximation rates of DeepONets for learning operators arising from advection–diffusion equations, *Neural Netw.* 153 (2022) 411–426.
- [10] Chensen Lin, Zhen Li, Lu Lu, Shengze Cai, Martin Maxey, George Em Karniadakis, Operator learning for predicting multiscale bubble growth dynamics, *J. Chem. Phys.* 154 (10) (2021) 104118.
- [11] Chensen Lin, Martin Maxey, Zhen Li, George Em Karniadakis, A seamless multiscale operator neural network for inferring bubble dynamics, *J. Fluid Mech.* 929 (2021) A18, <http://dx.doi.org/10.1017/jfm.2021.866>.
- [12] Somdatta Goswami, Minglang Yin, Yue Yu, George Em Karniadakis, A physics-informed variational DeepONet for predicting crack path in quasi-brittle materials, *Comput. Methods Appl. Mech. Engrg.* 391 (2022) 114587, <http://dx.doi.org/10.1016/j.cma.2022.114587>, <https://www.sciencedirect.com/science/article/pii/S004578252200010X>.
- [13] P Clark Di Leoni, Lu Lu, Charles Meneveau, George Karniadakis, Tamer A Zaki, Deeponet prediction of linear instability waves in high-speed boundary layers, 2021, arXiv preprint [arXiv:2105.08697](https://arxiv.org/abs/2105.08697).
- [14] Julian D. Osorio, Zhicheng Wang, George Karniadakis, Shengze Cai, Chrys Chrysostomidis, Mayank Panwar, Rob Hovsapien, Forecasting solar-thermal systems performance under transient operation using a data-driven machine learning approach based on the deep operator network architecture, *Energy Convers. Manage.* 252 (2021) <http://dx.doi.org/10.1016/j.enconman.2021.115063>, <https://www.osti.gov/biblio/1839596>.
- [15] Shengze Cai, Zhicheng Wang, Lu Lu, Tamer A Zaki, George Em Karniadakis, DeepM&Mnet: Inferring the electroconvection multiphysics fields based on operator approximation by neural networks, *J. Comput. Phys.* 436 (2021) 110296.
- [16] Zhiping Mao, Lu Lu, Olaf Marxen, Tamer A. Zaki, George Em Karniadakis, DeepM&Mnet for hypersonics: Predicting the coupled flow and finite-rate chemistry behind a normal shock using neural-network approximation of operators, *J. Comput. Phys.* 447 (2021) 110698, <http://dx.doi.org/10.1016/j.jcp.2021.110698>, <https://www.sciencedirect.com/science/article/pii/S0021999121005933>.
- [17] Minglang Yin, Enrui Zhang, Yue Yu, George Em Karniadakis, Interfacing finite elements with deep neural operators for fast multiscale modeling of mechanics problems, *Comput. Methods Appl. Mech. Engrg.* (2022) 115027.
- [18] Pengzhan Jin, Shuai Meng, Lu Lu, MIONet: Learning multiple-input operators via tensor product, 2022, arXiv preprint [arXiv:2202.06137](https://arxiv.org/abs/2202.06137).
- [19] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, George Em Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data, *Comput. Methods Appl. Mech. Engrg.* 393 (2022) 114778.
- [20] Sifan Wang, Hanwen Wang, Paris Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed DeepONets, *Sci. Adv.* 7 (40) (2021) eabi8605.
- [21] Lu Lu, Raphaël Pestourie, Steven G Johnson, Giuseppe Romano, Multifidelity deep neural operators for efficient learning of partial differential equations with application to fast inverse design of nanoscale heat transport, 2022, arXiv preprint [arXiv:2204.06684](https://arxiv.org/abs/2204.06684).
- [22] Amanda A Howard, Mauro Perego, George E Karniadakis, Panos Stinis, Multifidelity deep operator networks, 2022, arXiv preprint [arXiv:2204.09157](https://arxiv.org/abs/2204.09157).
- [23] Subhayan De, Malik Hassanaly, Matthew Reynolds, Ryan N King, Alireza Doostan, Bi-fidelity modeling of uncertain and partially unknown systems using DeepONets, 2022, arXiv preprint [arXiv:2204.00997](https://arxiv.org/abs/2204.00997).
- [24] Lizuo Liu, Wei Cai, Multiscale DeepONet for nonlinear operators in oscillatory function spaces for building seismic wave responses, 2021, arXiv preprint [arXiv:2111.04860](https://arxiv.org/abs/2111.04860).
- [25] Guang Lin, Christian Moya, Zecheng Zhang, Accelerated replica exchange stochastic gradient langevin diffusion enhanced Bayesian DeepONet for solving noisy parametric PDEs, 2021, arXiv preprint [arXiv:2111.02484](https://arxiv.org/abs/2111.02484).
- [26] Apostolos F Psaros, Xuhui Meng, Zongren Zou, Ling Guo, George Em Karniadakis, Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons, 2022, arXiv preprint [arXiv:2201.07766](https://arxiv.org/abs/2201.07766).
- [27] Yibo Yang, Georgios Kissas, Paris Perdikaris, Scalable uncertainty quantification for deep operator networks using randomized priors, *ArXiv E-Prints* (2022) [arXiv:2203.03048](https://arxiv.org/abs/2203.03048).
- [28] Christian Moya, Shiqi Zhang, Meng Yue, Guang Lin, DeepONet-Grid-UQ: A trustworthy deep operator framework for predicting the power grid's post-fault trajectories, 2022, arXiv preprint [arXiv:2202.07176](https://arxiv.org/abs/2202.07176).
- [29] E. Barnard, L.F.A. Wessels, Extrapolation and interpolation in neural network classifiers, *IEEE Control Syst. Mag.* 12 (5) (1992) 50–53, <http://dx.doi.org/10.1109/37.158898>.
- [30] Keyulu Xu, Mozhi Zhang, Jingling Li, Simon S Du, Ken-ichi Kawarabayashi, Stefanie Jegelka, How neural networks extrapolate: From feedforward to graph neural networks, 2020, arXiv preprint [arXiv:2009.11848](https://arxiv.org/abs/2009.11848).
- [31] Georgios Kissas, Jacob Seidman, Leonardo Ferreira Guilhoto, Victor M Preciado, George J Pappas, Paris Perdikaris, Learning operators with coupled attention, 2022, arXiv preprint [arXiv:2201.01032](https://arxiv.org/abs/2201.01032).

- [32] Xin-Yang Liu, Hao Sun, Min Zhu, Lu Lu, Jian-Xun Wang, Predicting parametric spatiotemporal dynamics by multi-resolution PDE structure-preserved deep learning, 2022, <http://dx.doi.org/10.48550/ARXIV.2205.03990>, arXiv preprint arXiv:2205.03990.
- [33] Marc C. Kennedy, Anthony O'Hagan, Predicting the output from a complex computer code when fast approximations are available, *Biometrika* 87 (1) (2000) 1–13.
- [34] András Sobester, Alexander Forrester, Andy Keane, *Engineering Design Via Surrogate Modelling: A Practical Guide*, John Wiley & Sons, 2008.
- [35] Xuhui Meng, George Em Karniadakis, A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse PDE problems, *J. Comput. Phys.* 401 (2020) 109020.
- [36] Lu Lu, Ming Dao, Punit Kumar, Upadrasta Ramamurty, George Em Karniadakis, Subra Suresh, Extraction of mechanical properties of materials through deep learning from instrumented indentation, *Proc. Natl. Acad. Sci.* 117 (13) (2020) 7052–7062.
- [37] Lu Lu, Ming Dao, Subra Suresh, George Karniadakis, Machine Learning Techniques for Estimating Mechanical Properties of Materials, Google Patents, 2022, US Patent App. 17/620, 219.
- [38] Sinno Jialin Pan, Qiang Yang, A survey on transfer learning, *IEEE Trans. Knowl. Data Eng.* 22 (10) (2009) 1345–1359.
- [39] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, Qing He, A comprehensive survey on transfer learning, *Proc. IEEE* 109 (1) (2020) 43–76.
- [40] Yixiang Deng, Lu Lu, Laura Aponte, Angeliki M Angelidi, Vera Novak, George Em Karniadakis, Christos S Mantzoros, Deep transfer learning and data augmentation improve glucose levels prediction in type 2 diabetes patients, *NPJ Digital Medicine* 4 (1) (2021) 1–13.
- [41] Jiang Lu, Pinghua Gong, Jieping Ye, Changshui Zhang, Learning from very few samples: A survey, 2020, arXiv preprint arXiv:2009.02653.
- [42] Yaqing Wang, Quanming Yao, James T. Kwok, Lionel M. Ni, Generalizing from a few examples: A survey on few-shot learning, *ACM Comput. Surv.* 53 (3) (2020) 1–34.
- [43] Somdatta Goswami, Katiana Kontolati, Michael D Shields, George Em Karniadakis, Deep transfer learning for partial differential equations under conditional shift with DeepONet, 2022, arXiv preprint arXiv:2204.09810.
- [44] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Fourier neural operator for parametric partial differential equations, 2020, arXiv preprint arXiv:2010.08895.
- [45] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Neural operator: Graph kernel network for partial differential equations, 2020, arXiv preprint arXiv:2003.03485.
- [46] Huaiqian You, Yue Yu, Marta D'Elia, Tian Gao, Stewart Silling, Nonlocal kernel network (NKN): a stable and resolution-independent deep neural network, 2022, arXiv preprint arXiv:2201.02217.
- [47] Nathaniel Trask, Ravi G Patel, Ben J Gross, Paul J Atzberger, GMLS-Nets: A framework for learning from unstructured data, 2019, arXiv preprint arXiv:1909.05371.
- [48] Ravi G Patel, Nathaniel A Trask, Mitchell A Wood, Eric C Cyr, A physics-informed operator regression framework for extracting data-driven continuum models, *Comput. Methods Appl. Mech. Engrg.* 373 (2021) 113500.
- [49] Matthias Gelbrich, On a formula for the L2 Wasserstein metric between measures on Euclidean and Hilbert spaces, *Math. Nachr.* 147 (1) (1990) 185–203.
- [50] Ameya D. Jagtap, Kenji Kawaguchi, George Em Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks, *Proc. R. Soc. Lond. Ser. A Math. Phys. Eng. Sci.* 476 (2239) (2020) 20200334.
- [51] Ameya D Jagtap, Yeonjong Shin, Kenji Kawaguchi, George Em Karniadakis, Deep kronecker neural networks: A general framework for neural networks with adaptive activation functions, *Neurocomputing* 468 (2022) 165–180.
- [52] Stuart Geman, Elie Bienenstock, René Doursat, Neural networks and the bias/variance dilemma, *Neural Comput.* 4 (1) (1992) 1–58.
- [53] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, Jerome H Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, vol. 2, Springer, 2009.
- [54] Mikhail Belkin, Daniel Hsu, Siyuan Ma, Soumik Mandal, Reconciling modern machine-learning practice and the classical bias–variance trade-off, *Proc. Natl. Acad. Sci.* 116 (32) (2019) 15849–15854.
- [55] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, Ilya Sutskever, Deep double descent: Where bigger models and more data hurt, *J. Stat. Mech. Theory Exp.* 2021 (12) (2021) 124003.
- [56] Vinod Nair, Geoffrey E. Hinton, Rectified linear units improve restricted Boltzmann machines, in: *ICML*, 2010.
- [57] Stefan Elfving, Eiji Uchibe, Kenji Doya, Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, *Neural Netw.* 107 (2018) 3–11.
- [58] Dan Hendrycks, Kevin Gimpel, Gaussian error linear units (GeLUs), 2016, arXiv preprint arXiv:1606.08415.
- [59] Qingguo Hong, Qinyang Tan, Jonathan W Siegel, Jinchao Xu, On the activation function dependence of the spectral bias of neural networks, 2022, arXiv preprint arXiv:2208.04924.
- [60] Pengzhan Jin, Lu Lu, Yifa Tang, George Em Karniadakis, Quantifying the generalization error in deep learning in terms of data distribution and neural network smoothness, *Neural Netw.* 130 (2020) 85–99.
- [61] Ian J Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, Yoshua Bengio, An empirical investigation of catastrophic forgetting in gradient-based neural networks, 2013, arXiv preprint arXiv:1312.6211.
- [62] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al., Overcoming catastrophic forgetting in neural networks, *Proc. Natl. Acad. Sci.* 114 (13) (2017) 3521–3526.
- [63] Sifan Wang, Hanwen Wang, Paris Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed DeepONets, *Sci. Adv.* 7 (40) (2021) eabi8605, <http://dx.doi.org/10.1126/sciadv.abi8605>, arXiv:https://www.science.org/doi/pdf/10.1126/sciadv.abi8605.

- [64] Samuel Lanthaler, Siddhartha Mishra, George E. Karniadakis, Error estimates for DeepONets: A deep learning framework in infinite dimensions, *Trans. Math. Appl* 6 (1) (2022) tnac001.
- [65] Tim De Ryck, Siddhartha Mishra, Generic bounds on the approximation error for physics-informed (and) operator learning, 2022, arXiv preprint [arXiv:2205.11393](https://arxiv.org/abs/2205.11393).
- [66] Nikola Kovachki, Samuel Lanthaler, Siddhartha Mishra, On universal approximation and error bounds for Fourier neural operators, *J. Mach. Learn. Res.* 22 (2021) Art–No.
- [67] Carlo Marcati, Christoph Schwab, Exponential convergence of deep operator networks for elliptic partial differential equations, 2021, arXiv preprint [arXiv:2112.08125](https://arxiv.org/abs/2112.08125).
- [68] Lukas Herrmann, Christoph Schwab, Jakob Zech, Neural and gpc operator surrogates: construction and expression rate bounds, 2022, arXiv preprint [arXiv:2207.04950](https://arxiv.org/abs/2207.04950).
- [69] Christoph Schwab, Andreas Stein, Deep solution operators for variational inequalities via proximal neural networks, *Res. Math. Sci* 9 (3) (2022) 1–35.
- [70] Maarten V de Hoop, Nikola B Kovachki, Nicholas H Nelsen, Andrew M Stuart, Convergence rates for learning linear operators from noisy data, 2021, arXiv preprint [arXiv:2108.12515](https://arxiv.org/abs/2108.12515).